

班 级     2018039

学 号     20009200405

西安电子科技大学

# 本科毕业设计论文



题 目     个人隐私信息删除指令

的可信通知与转发研究

学 院     网络与信息安全学院

专 业     网络空间安全

学生姓名     朱星岳

导师姓名     宋祁朋



## 摘 要

在当今数据资源极大丰富的信息时代，数据跨域共享为数字产业的发展提供了无限可能。然而，这也对个人信息权益保障提出了挑战。隐私泄露等事件屡见不鲜，社会各界愈发认识到保障删除权的必要性。对此，世界各主要经济体均以立法的形式，明确公民依法享有删除权。

然而，删除权在相关技术层面的进展严重滞后于立法层面的进度。从技术角度出发，**保障删除通知在不可信数据控制者网络中分发的完备性与可信性对于保护个人的删除权利至关重要**。具体而言，跨域删除通知分发方案必须解决以下挑战：1). 完备性，旨在确保所有应该接收删除通知的数据控制者按预期接收到通知。2). 可信性，旨在确保删除通知分发过程的完整性和安全性，使其免受篡改、重放，冒充等恶意攻击行为的影响。3). 轻量级，尽可能降低方案的整体开销，有助于提高数据控制者对该方案的接受度。

本毕设在生态架构层面，提出了一种跨层协同中心化管理架构，由监管平面与业务平面构成，监管平面为监管机构提供支持，负责记录各数据控制者之间的数据共享传播历史。业务平面涵盖了处理用户删除请求的广大数据控制者。其次，在通信协议层面，设计了删除通知可信分发协议，将删除通知转发问题形式化为个人信息流转树双向遍历验证问题，在可信执行环境（TEE）的支持下，采用逐跳转发、前向遍历验证、递归反向确认与自适应概率采样验证进行通知工作。在工程实现层面，基于前述协议开发了一套删除通知可信分发系统，该系统通过前后端协同工作，能够模拟真实删除场景下的删除通知可信分发业务流程。

实验与测试结果表明，所提出的架构方案能够使监管机构和数据控制者在合理的成本内共同解决删除通知可信分发的的问题。删除通知可信分发协议能够以轻量级且健壮的方式可信分发删除通知，使得数据控制者与用户免受威胁模型中潜在恶意攻击行为的影响。所开发的删除通知可信分发系统能够根据数据量的增长进行动态适应，以高效处理大规模删除通知流量。同时系统还提供了直观的可视化界面，能够清晰展示删除通知可信分发的整体流程。

**关键词：**删除权      删除通知分发与确认      隐私保护



## ABSTRACT

In the era of abundant data resources in the information age, cross-domain data sharing has provided endless possibilities for the development of the digital industry. However, it also poses challenges to the protection of individual privacy rights. Privacy breaches and similar incidents have become increasingly common, leading society to recognize the necessity of safeguarding the right to deletion. In response, major economies around the world have legislated to clearly establish the right to deletion for citizens.

However, the progress of technology in ensuring the right to deletion lags far behind the pace of legislation. From a technological perspective, ensuring the completeness and trustworthiness of the distribution of deletion notices in an untrusted data controller network is crucial for protecting individuals' deletion rights. Specifically, a cross-domain deletion notice distribution scheme must address the following challenges: Completeness, Trustworthiness and Lightweightness.

In this thesis, a centralized management architecture that leverages cross-layer collaboration is proposed at the ecosystem architecture level. It consists of a regulatory plane and a business plane, where the regulatory plane supports regulatory agencies and is responsible for recording the data sharing propagation history among data controllers. The business plane encompasses a wide range of data controllers involved in processing user deletion requests. Furthermore, a trusted deletion notice distribution protocol is designed at the communication protocol level. It formalizes the forwarding of deletion notices as a bidirectional traversal verification problem in the personal information flow tree. With the support of a trusted execution environment (TEE), it employs hop-by-hop forwarding, forward traversal verification, recursive backward confirmation, and adaptive probabilistic sampling for notice distribution. At the engineering implementation level, a trusted deletion notice distribution system is developed based on the aforementioned protocol. This system, through collaboration between the front-end and back-end, simulates the trustworthy distribution process of deletion notices in real-world scenarios.

Experimental and testing results demonstrate that the proposed architectural scheme enables regulatory agencies and data controllers to jointly address the problem of trusted deletion notice distribution at reasonable costs. The deletion notice distribution protocol achieves trustworthy distribution of deletion notices in a lightweight and robust manner, protecting data controllers and users from potential malicious attacks outlined in the threat model. The developed deletion notice distribution system dynamically adapts to the growth of data volume, efficiently handling large-scale deletion notice traffic. Additionally, the system provides an intuitive visual interface that clearly illustrates the overall process of deletion notice distribution.

**Keywords:** Right to deletion      Deletion notification distribution and acknowledgement      Privacy protection



# 目 录

|                      |           |
|----------------------|-----------|
| <b>第一章 绪论</b>        | <b>1</b>  |
| 1.1 选题背景及研究意义        | 1         |
| 1.2 国内外研究现状          | 4         |
| 1.2.1 学术界删除问题研究现状    | 4         |
| 1.2.1.1 不可恢复的数据删除    | 4         |
| 1.2.1.2 可验证数据删除      | 5         |
| 1.2.2 产业界删除系统开发现状    | 6         |
| 1.2.3 国内外研究现状总结      | 7         |
| 1.3 主要贡献             | 7         |
| 1.4 论文结构安排           | 8         |
| <b>第二章 问题描述及技术理论</b> | <b>9</b>  |
| 2.1 删除通知分发流程         | 9         |
| 2.2 威胁模型             | 10        |
| 2.3 理论技术             | 12        |
| 2.3.1 HMAC           | 12        |
| 2.3.2 Mininet 网络仿真器  | 13        |
| 2.3.3 可信执行环境 (TEE)   | 13        |
| 2.3.4 JMeter 性能测试工具  | 14        |
| <b>第三章 方案设计</b>      | <b>15</b> |
| 3.1 删除通知可信分发系统设计     | 15        |
| 3.1.1 系统总体设计         | 15        |
| 3.1.2 系统功能           | 17        |
| 3.2 删除通知可信分发协议       | 18        |
| 3.2.1 前向遍历式验证机制      | 18        |
| 3.2.2 反向递归式确认机制      | 20        |
| 3.2.3 自适应概率抽样验证      | 22        |
| 3.2.4 删除通知分发网络功能模块   | 23        |
| 3.3 协议流程             | 24        |

|            |                           |           |
|------------|---------------------------|-----------|
| 3.3.1      | 系统模型 . . . . .            | 24        |
| 3.3.2      | 隐私信息初始化流程 . . . . .       | 24        |
| 3.3.3      | 删除通知可信分发流程 . . . . .      | 26        |
| 3.3.3.1    | 删除通知消息生成算法 . . . . .      | 27        |
| 3.3.3.2    | 删除通知消息解析签名算法 . . . . .    | 29        |
| 3.3.3.3    | 删除反馈消息生成算法 . . . . .      | 31        |
| <b>第四章</b> | <b>工程实现 . . . . .</b>     | <b>33</b> |
| 4.1        | 开发业务流程 . . . . .          | 33        |
| 4.2        | 开发技术选型 . . . . .          | 34        |
| 4.3        | 前端实现 . . . . .            | 36        |
| 4.3.1      | 前端开发架构 . . . . .          | 36        |
| 4.3.2      | 代码实现架构 . . . . .          | 37        |
| 4.4        | 后端实现 . . . . .            | 38        |
| 4.4.1      | 删除指令通知与确认系统 . . . . .     | 41        |
| 4.4.2      | 隐私数据流转状态管理与存证系统 . . . . . | 43        |
| 4.5        | 数据库设计 . . . . .           | 45        |
| 4.6        | 消息类 . . . . .             | 48        |
| <b>第五章</b> | <b>系统测试 . . . . .</b>     | <b>51</b> |
| 5.1        | 安全性分析 . . . . .           | 51        |
| 5.2        | 删除通知可信分发协议性能分析 . . . . .  | 52        |
| 5.2.1      | 实验测试环境 . . . . .          | 52        |
| 5.2.2      | 性能评估 . . . . .            | 52        |
| 5.3        | 删除通知可信分发系统测试 . . . . .    | 55        |
| 5.3.1      | 测试数据 . . . . .            | 55        |
| 5.3.1.1    | 测试数据设计 . . . . .          | 55        |
| 5.3.1.2    | 测试数据生成 . . . . .          | 57        |
| 5.3.2      | 性能测试 . . . . .            | 58        |
| 5.3.3      | 系统功能测试 . . . . .          | 61        |
| 5.3.3.1    | 用户操作端 . . . . .           | 62        |
| 5.3.3.2    | 企业操作端 . . . . .           | 65        |

|                               |           |
|-------------------------------|-----------|
| 5.3.3.3 中心监管机构端 . . . . .     | 68        |
| 5.3.3.4 删除通知分发流程性测试 . . . . . | 69        |
| <b>第六章 总结与展望 . . . . .</b>    | <b>73</b> |
| 6.1 总结 . . . . .              | 73        |
| 6.2 展望 . . . . .              | 74        |
| <b>致谢 . . . . .</b>           | <b>75</b> |
| <b>参考文献 . . . . .</b>         | <b>77</b> |



## 第一章 绪论

本章分析了本次毕设的选题背景及研究意义，对于整个毕设相关领域的国内外研究现状进行了阐述，最后对于本毕设的研究内容及论文结构分别进行了说明。

### 1.1 选题背景及研究意义

当前时代背景下，以大数据、人工智能与云计算等技术为代表，世界正迈入一个信息资源极大丰富的大数据时代。数据更是被视为一种宝贵的资源，一定程度上有助于为个人提供更具针对性的服务，更为社交、商业与数字产业的蓬勃发展提供了无限可能。然而，这同样对个人信息权益保障提出了挑战。隐私数据大规模泄露等事件屡见不鲜，为网络诈骗等不法行为提供便利，进而侵犯了公民的合法权益。社会各界愈发认识到对存储在互联网平台的个人隐私信息进行删除的必要性。



图 1.1 世界各主要经济体隐私保护法案一览

对此，近年来世界各主要经济体均通过立法等形式明确规定公民依法享有删除权。欧盟的《通用数据保护条例》(GDPR)<sup>[1]</sup>、加州的《消费者隐私法》(CCPA)<sup>[2]</sup>、中国的《个人信息保护法》<sup>[3]</sup>等多项法律法规都规定，数据控制者（即，公司、数字平台、机构等）有义务保护个人的“被遗忘权”（又称删除权、擦除权）。删除权是指在符合法律规定或者当事人约定的情形下，信息主体可以请求信息处理者及时删除相关个人信息权利，旨在保障信息主体对其个人信息的自主决定权<sup>[4]</sup>。

然而，删除权在相关技术层面的发展远远滞后于立法层面的进度。在实际生活中，公民的删除权难以得到有效保障。

一方面，从企业层面角度出发，在个人删除请求满足特定条件的情况下，数据控制者需遵守法律要求及时删除个人数据。尽管如此，目前工业界尚未形成一套成熟的隐私数据删除技术方案与隐私数据删除权保障系统，致使企业在实际运营中面临着巨大的合规风险。且企业保障用户删除权的主观意愿与行事动力不强，在开发与维护成熟的跨域完备删除通知系统上所投入的人力物力有限。

另一方面，从技术保障层面出发，保障删除权主要面临两大难题：1). 单个数据控制者内可验证的删除，即如何永久地删除目标数据<sup>[5][6]</sup>，以及如何证明目标数据已被永久删除<sup>[7][8]</sup>。2). 数据控制者之间应如何安全可信的通知并分发个人隐私数据。实际上，单个数据控制域内的数据彻底删除问题已引起了广泛的关注，这主要是因为随着数据保护法规的日益严格，数据控制者面临一定的合规压力，需要确保自身有效地响应个人数据删除的请求。然而，跨节点间删除指令可信分发问题，目前还鲜有研究与讨论，个人隐私信息跨平台传播力强，数据流转涉及面广，相关隐私信息流向极难追踪。如何在数据控制者间安全地分发删除通知，为个人删除权提供保障，是构建删除权网络基础设施必不可少的部分，也是本毕设的讨论重点。



图 1.2 数据跨域流转

设计一种安全可信的删除通知分发机制是一项颇有挑战的任务。个人隐私数据可在数据控制者之间跨域流通（参见图 1.2）。首先，为确保持有待删除数据的所有数据控制者得到正确通知，高效地记录不同数据控制者之间的数据流通历史至关重要。这能够确保数据被彻底删除，并在删除过程中提供透明度与可追溯性，同时降低数据控制者各自的合规操作成本。其次，由于媒体报道、隐私倡导者的努力以及诸多侵犯个人隐私信息的网络公开事件，公民对删除权的认识日益增强，预计数据控制者将接收到大量删除请求。考虑到遵守删除权本身并不产生经济利益，只有轻量级且安全的删除通知分发机制才更容易被数据控制者接受，以间接保障公民的删除权利。

为应对上述挑战，本毕设在**生态架构层面、通信协议层面以及工程实现层面**作出突破。

**在生态架构层面**，将删除权保障网络基础设施分为监管平面与业务平面，提出了监管平面与业务平面之间的跨层协同解决方案。其核心思想是：在监管平面，高效地记录数据控制者之间的数据流转历史，当用户行使删除权时，系统能够以低开销重构待删除数据的数据流转历史，从而精确地指导删除通知在相关数据控制者之间的正确分发，并确保通知过程的完整性。在业务平面，设计了一套既轻量又安全可信的删除通知分发协议，利用消息验证码及可信执行环境（TEE）等技术，以可接受的资源开销，保障删除通知分发过程的可信性。

**在通信协议层面**，本毕设设计并实现了一套删除传播全过程安全可信、可验证的删除通知协议。将删除通知的分发过程建模为一颗以率先收到用户删除请求的数据控制者为根节点的生成树，并进一步将删除通知分发的过程形式化描述为生成树双向遍历验证问题。

**在工程实现层面**，本毕设开发了一套删除通知可信分发系统，该系统由**删除指令通知与确认系统、隐私信息流转记录存证系统以及隐私删除可视化平台**等子系统组成。在前端实现部分，考虑在隐私删除可视化平台之上设置用户端，企业端与中心监管机构端这三种交互界面，以模拟现实情景下的删除通知可信分发业务流程。在后端实现部分，使删除指令通知与确认系统以及隐私信息流转记录存证系统协作运行，共同遵循删除通知可信分发协议的规定，以确保分发过程的可信性和完备性。

经测试，本系统能够以合理的开销，保障用户行使删除权过程中的删除通知

在不同数据控制者之间分发的完备性以及可信性。支持在大流量、高并发环境下处理跨域删除通知任务。旨在为公民的信息删除权提供必要的技术支持，填补删除通知领域的工程空白，从而保障公民的删除权，并为企业开发者提供一套完整的删除权技术保障方案。

## 1.2 国内外研究现状

近年来，社会各界愈发认识到删除权的重要性，关于保障删除权的研究引起了广泛关注。本小节将从产业界删除系统与学术界删除研究两个方向入手，对现有的研究工作进行总结。

### 1.2.1 学术界删除问题研究现状

#### 1.2.1.1 不可恢复的数据删除

许多研究人员探讨了**从物理介质中不可恢复地擦除数据的方法**。Reardon 等学者<sup>[9]</sup>研究了不同接口到物理介质的不可恢复数据删除方法，对不同能力的敌手进行了分类，并系统化了对安全删除技术特性的研究。Lee 等学者<sup>[10]</sup>设计了具有安全删除功能的 NAND 闪存文件系统，修改了 YAFFS，通过加密来实现文件删除，并强制将特定文件的所有密钥存储在一起，只需一次擦除，即可安全删除全部文件。莫臻等学者<sup>[11]</sup>提出了一种云存储数据安全删除方案，通过密钥调制技术实现细粒度数据删除，避免了对第三方服务器的依赖，简化了密钥管理。熊金波等学者<sup>[12]</sup>提出了一种基于密钥策略，带有时间属性的属性加密技术 (KP-TSABE)，这是一种针对云计算环境中数据的安全自毁方案，当到达用户指定的过期时间时，敏感数据将自我销毁。杨力等学者<sup>[13]</sup>在移动设备的安全数据删除领域做出了贡献，他们提出了一种利用用户空间文件系统在文件级别上提供安全删除保障的方法，这一方法与底层文件系统以及存储介质的特性无关。熊金波等学者<sup>[5]</sup>同样还提出了一种新颖的密钥派生加密 (KDE) 算法以及基于此算法的安全数据删除 (SDDK) 方案，通过设计一个基于闪存层次的节点密钥树和 KDE 算法，在简化了密钥管理的同时加密了用户敏感数据。该 SDDK 方案结合了部分块擦除与密钥删除技术，实现了 IoT 设备上数据到期后的安全删除。Gnatyuk 等学者<sup>[14]</sup>调查了不同国家的数据删除标准，并提出了基于随机覆写的 STM Shredder 方法，以确保数据删除的可靠性。数据消毒会在闪存存储介质上留下各种数据“残留”，这些残留可以用于恢复已删除的数据。RedFlash<sup>[15]</sup>是一项用于在闪存存储器上进行数据删除，同时删除其对应重复数据的技术。其研究目标是确保删除操作是不可恢复的，即使这会

略微降低性能。

### 1.2.1.2 可验证数据删除

此研究方向旨在为数据所有者验证所需数据是否已被彻底删除。**可验证删除**研究中的主要方法包括加密密钥删除、覆写技术和区块链技术等策略。

**加密密钥删除**的核心思想是彻底地销毁用于加密待删除数据的密钥，使得加密数据无法被访问。在此方面，薛靓等学者<sup>[16]</sup>提出了一种可靠的云数据加密数据删除方案，该方案基于属性基密钥策略加密（AD-KP-ABE），这种方法不仅有助于进行细粒度的数据访问，同时也确保了数据被可靠删除。郑东等学者<sup>[17]</sup>阐述了一个用于云存储的可信删除系统模型，结合了可验证删除技术，并提出了各种及时与细粒度的删除验证方法。马骏等学者<sup>[18]</sup>提出了一种使用密文策略属性加密（CP-ABE）技术的方案，通过构建属性关联树，实现了云数据的细化安全删除。此外，该方案引入了一种规则替换算法，增强了删除过程的验证能力，提高了数据删除验证的安全性与可靠性。田军峰等学者<sup>[19]</sup>提出了一种结合多级安全原则与动态技术的策略，以改进访问控制，能够有效管理数据删除，并利用默克尔哈希树提供可验证的删除证明。

在**基于覆写的删除验证**研究中，数据所有者生成随机数据块并发送给服务器。服务器使用这些随机块覆写待删除的数据，并在覆写后为每个数据块生成一个验证值。例如，Bauer 等学者<sup>[20]</sup>提出了一个实现了 ext2 文件系统的方法，通过使用异步覆写，牺牲了即时安全性，但最终实现了更可用以及完整的安全删除。Bhat 等学者<sup>[21]</sup>提出了一种可靠高效的堆叠文件系统 restFS，以解决现有透明安全文件删除系统扩展的可靠性与效率不足的问题。与现有技术中的文件级覆写不同，restFS 通过块级覆写提高了可靠性和效率。杜瑞忠等学者<sup>[22]</sup>提出了一种基于密文重新加密和覆写验证（WV-CP-ABE）的方案，通过更改访问控制策略的密文来执行数据的细化删除，并使用基于脏数据块覆写的可搜索路径哈希二叉树（DSMHT）验证待删除数据的正确性。杨昌松等学者<sup>[23]</sup>提出了一种基于可逆 Bloom 过滤器的高效细粒度外包数据删除方案，不仅能够实现存储和删除结果的公开和私有可验证性，还允许用户在云服务器不诚实地维护/删除数据和生成相应证据时以高概率轻松检测到其恶意行为。

**区块链技术**作为一种有前景的解决方案，能够实现可验证的数据删除，无需依赖于任何可信第三方实体，从而保证删除操作的效力和可追溯性。例如，杨昌松

等学者同样<sup>[24]</sup>介绍了一种基于区块链的方案,允许公开验证个人信息的删除。在这个框架中,进行数据删除操作后,会生成删除凭证并记录在区块链上,从而确保数据删除的可验证性。

### 1.2.2 产业界删除系统开发现状

随着个人信息保护法的出台,大部分数据控制者出于数据合规压力与相关法律要求,在隐私政策中均声明了会保障用户的隐私删除权。在这部分中,我们选取了市面上一些主流应用进行了调研。结合该应用隐私政策与相关法律法规,参照个人信息保护法<sup>[3]</sup>与 GDPR<sup>[1]</sup>中的部分规定,对各应用内置的隐私删除系统进行评估,并对**产业界删除权保障系统现状**进行总结。

表 1.1 主流产业界应用删除权保障现状

| 应用名称 | 隐私政策             | 删除方式             | 便捷程度            | 跨域删除 |
|------|------------------|------------------|-----------------|------|
| 微信   | 隐私政策含有支持删除权举措    | 手动申请             | 删除渠道偏僻难寻且对用户不友好 | 不支持  |
| 百度搜索 | 隐私政策中声明支持个人信息保护法 | 以账号注销代行用户隐私信息删除权 | 未设置用户隐私信息删除渠道   | 不支持  |
| 淘宝   | 隐私政策中声明支持个人信息保护法 | 以账号注销代行用户隐私信息删除权 | 未设置用户隐私信息删除渠道   | 不支持  |
| 知乎   | 隐私政策中声明支持个人信息保护法 | 手动申请             | 删除渠道偏僻难寻且对用户不友好 | 不支持  |
| 抖音   | 隐私政策中声明支持个人信息保护法 | 手动申请             | 删除渠道偏僻难寻且对用户不友好 | 不支持  |
| 京东   | 隐私政策中声明支持个人信息保护法 | 以账号注销代行用户隐私信息删除权 | 未设置用户隐私信息删除渠道   | 不支持  |

从整体调研结果来看,主流产业界的删除权保障系统大都存在以下现象:

1. **支持用户删除权:** 各互联网产商其删除系统均在隐私政策中声明了支持个人信息删除权。
2. **删除功能不透明:** 删除功能不够透明,删除渠道偏僻难寻,删除操作对用户不友好。
3. **不支持跨域删除:** 所有系统仅支持在已有数据域中清除用户隐私信息,均不

支持通知第三方数据处理者删除相关隐私信息。

**以微信为例**，微信为第三方小程序提供了个人信息获取接口，第三方小程序在征得个人信息主体同意的前提下，可从微信获取指定范围的个人信息。当个人信息主体注销微信账号时，微信将按法律法规留存个人信息，超出必要保存期限后，将对个人信息进行删除。然而，微信并未提供个人信息删除通知机制通知第三方小程序删除个人信息。

**以某搜索引擎服务为例**，当个人信息主体需要删除搜索结果中关于自身的个人信息时，需自行寻找所有相关个人信息链接，手动填写申请并附上所有待删除的链接，然后由科技公司工作人员手动核验删除请求的合规性与合法性。

### 1.2.3 国内外研究现状总结

综上所述，在**产业界删除通知可信分发系统的研发方面**，即使各大互联网企业均在其产品中集成了支持个人信息删除权的信息删除功能。但现有技术方案的自动化、智能化程度尚显不足，无法实现完整的个人信息删除通知机制，且疏于与相关法律法规进行联动，大多仅支持简单的增删查改，难以保障用户删除权，业界亟需一款能够进行删除通知可信分发的系统。在**学术界删除研究现状方面**，观察到现有关于删除权的研究主要集中在从物理介质中不可恢复地擦除数据以及可验证删除之上。对**跨域多数据控制者之间进行可信安全删除通知分发的研究较少**，这也是本毕设尝试努力解决的目标。

## 1.3 主要贡献

**1. 跨层协作集中式网络管理架构**：本毕设提出了一套跨层协作的集中式网络管理架构，支持删除指令的可信通知与分发。它由监管平面和业务平面组成。监管平面主要由中央监管机构组成，负责记录数据流通历史，并根据不同的删除请求构建相应的数据流转图；业务平面由众多数据控制者构成，负责响应用户删除请求，提供对应信息删除服务。该架构可以使监管机构与数据控制者各自以合理的开销，协同解决删除指令可信分发问题。

**2. 轻量级可信删除通知协议**：针对业务平面内的数据控制者，本毕设设计了一种轻量级可信删除通知协议。该协议采用逐跳前向遍历验证、递归后向确认和概率抽样验证机制，能够处理根据目标数据的流转历史建立的任意拓扑结构。同时借助可信执行环境（TEE，如 Intel 的 SGX，ARM 的 TrustZone）提供删除权限

保护机制。通过该协议，相关数据控制者可以安全地分发删除通知，以轻量级和健壮的方式免受威胁模型中潜在恶意攻击的影响。

**3. 删除通知可信分发系统：**本毕设实现了一款删除通知可信分发系统，旨在以低开销的资源成本，保障删除通知分发的完备性与可信性，致力于为隐私数据的个人权益保障体系提供删除权和被遗忘权的权益保障支撑，解决了传统删除验证方案不支持跨域删除、通信开销较大、计算成本较高、第三方机构存储负荷重等问题。删除通知可信分发系统由删除指令通知与确认系统，隐私数据流转状态存证系统以及隐私信息处理可视化平台三大子系统构成。

## 1.4 论文结构安排

本论文各个章节的内容安排如下：

第一章，本章为绪论部分，分析了本次毕设的选题背景和研究意义，对于整个毕设相关领域的国内外研究现状进行了阐述，最后对于本毕设的主要贡献及论文结构分别进行了说明。

第二章，本章为问题描述及技术理论，描述并分析了删除通知的基本分发流程与威胁模型。同时介绍了本毕设涉及的技术理论。

第三章，本章为方案设计，主要介绍了删除通知可信分发系统的总体设计方案以其功能，并详细描述了其构成子系统的设计与作用。最后介绍了删除通知可信分发通信协议，并逐一介绍了协议所用机制的原理细节。

第四章，本章为工程实现，简要概述了系统整体的开发流程和前后端技术选择，并逐一介绍了前端和后端各自的实现方法。最后提供了数据库设计方案和消息类定义方案。

第五章，本章为系统测试，首先进行了对删除通知可信分发协议的安全性分析与性能评估，简述了性能测试的实验环境，并对协议的性能测试结果进行了分析与总结。接下来，对删除通知可信分发系统开展了全方位测试，介绍了系统测试数据的设计与生成方案，以及性能测试的测试环境与结果分析。最后，以隐私信息可视化平台为例，对整个系统进行了功能正确性测试，以验证系统的各项功能是否符合设计要求。

第六章，本章为总结与展望，对本毕设的总体工作进行了总结，指出了现有工作存在的不足之处，对删除通知可信分发系统中亟待改进的部分进行了展望。

## 第二章 问题描述及技术理论

本章主要介绍删除通知的基础分发流程，将删除通知转发问题形式化为个人信息流转树遍历验证问题。以及分析这一信息转发模型下的威胁模型。最后介绍了本毕设所涉及的技术理论。

### 2.1 删除通知分发流程

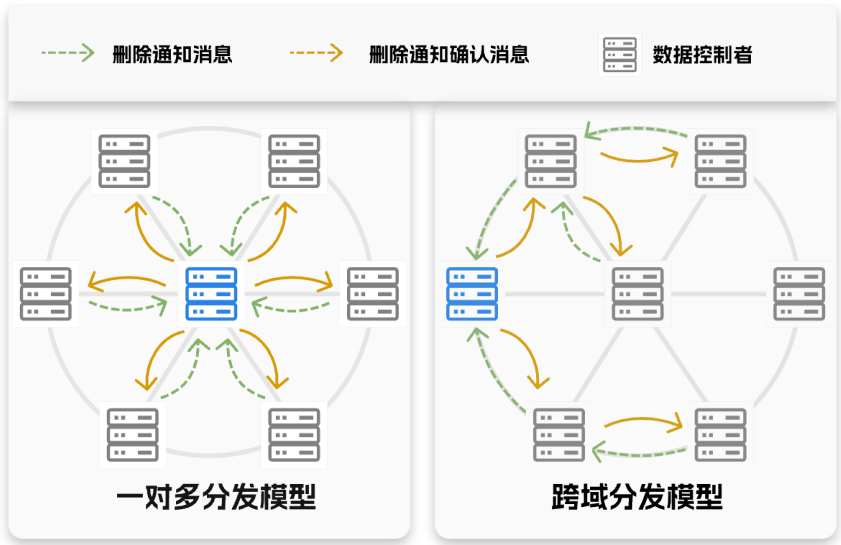


图 2.1 数据控制者间的删除通知分发模型

尽管个保法要求，当公民履行删除权时，所有共享目标数据的数据控制者，都有义务执行对应的删除操作。但现行法律法规并未明确规定应如何通知所有涉及的数据控制者。工业界和学术界都尚未就此提出解释。直观上，人们可能会认为在数据控制者之间分发删除通知的过程采用了简单直接的“一对多”模型，如图 2.1 所示，其中首个数据控制者（称为“源数据控制者”）收到删除请求后通知持有待删除数据的所有其他数据控制者。

然而，本毕设认为这种方法对源数据控制者增添了额外的性能负担。例如，数据控制者 A 与数据控制者 B 共享某些数据，而后者又与其他 1,000 个控制者共享该数据。当用户联系源数据控制者，请求删除该数据时，后者将被迫通知其他 1,001 个数据控制者删除目标数据。这显然是不公平的，因为数据控制者 A 独自承担了遵守删除权的成本，而其他数据控制者享受了共享数据的好处，却几乎不承担义务。该类型的删除通知分发模型难以被数据控制者接受。

有鉴于此，本毕设提出了一种更合理的删除通知在数据控制者间的分发模型。参考图 2.1 所示的模型。该模型以源数据控制者为根节点，将删除通知传递给所有已共享待删除数据的后继数据控制者，直到最终的叶节点得到通知。删除通知的路径最终形成以源数据控制者为根的生成树，涵盖了所有相关的数据控制者。这个生成树与待删除数据的历史流路径相符。在这个模型中，每个数据控制者都合理地参与到删除通知的分发过程中。相较于一对多模型，该模型减少了源节点因删除权合规而导致的资源开销，并体现了“谁受益，谁担责”的法律原则。

## 2.2 威胁模型

数据控制者在处理删除指令时受到法规的约束，可以合理地假设数据控制者有强烈的动机保护其客户的删除权利。原因有两个方面：一方面，如果数据控制者非法保留应该删除的数据，将面临财务处罚甚至法律诉讼；另一方面，现如今存在诸多技术可以让用户验证数据控制者是否实际删除了他们的个人数据（例如机器遗忘、可验证数据删除等）。因此，本毕设假设数据控制者是可信的，并且在收到删除通知时会忠实执行。此模型专注于外部攻击者对删除通知分发过程的威胁，包括了各种攻击行为。

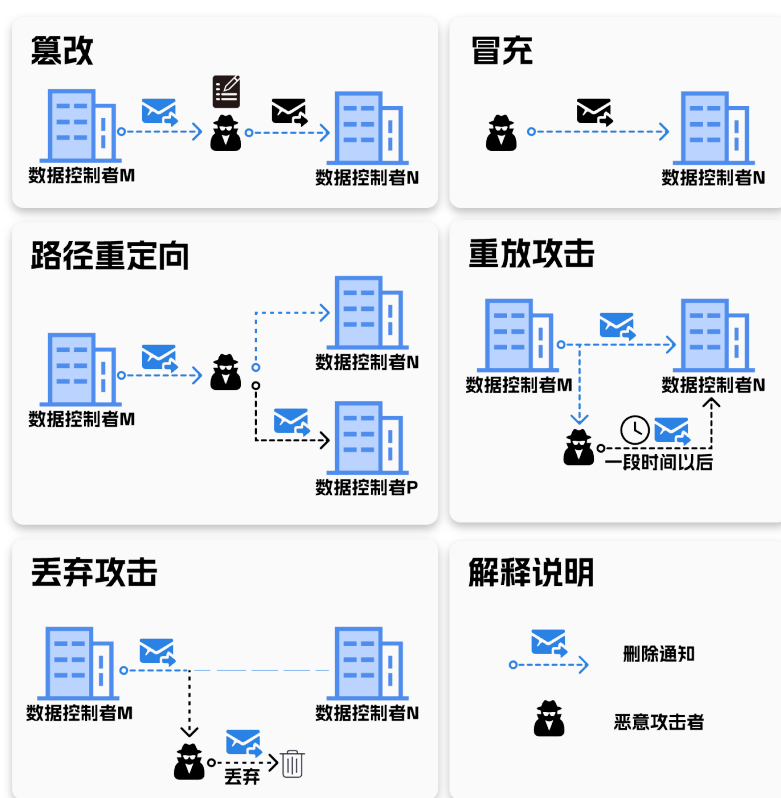


图 2.2 威胁模型

如图 2.2 所示, 对于跨数据控制者数据共享的情况, 执行删除权涉及大量的相关数据控制者。当删除通知通过不可信任的网络进行分发时, 恶意对手可能出于各种目的干扰删除通知的分发过程。删除通知在通过不可信任的网络进行分发时容易受到以下攻击行为的影响:

1. **篡改:** 对删除通知的篡改行为包括未经授权的修改, 改变删除通知的内容, 从而改变用户的删除意图。例如, Alice 打算在一个数据控制者那里删除她的个人数据, 数据控制者向其他存有 Alice 隐私信息的数据控制者发送了一个删除通知, 恶意攻击者 Eve 截获了该删除通知, 并将 Alice 的身份修改为 Bob 的身份, 最终导致 Bob 的信息被删除。
2. **冒充:** 冒充是指恶意攻击者 Eve 伪装成目标数据控制者的上一跳数据控制者的情况, 欺骗目标数据控制者在未经授权的情况下执行恶意删除通知。例如, 恶意攻击者 Eve 获取了 Alice 的 ID, 然后冒充 Alice 个人信息流通树中的某一个数据控制者, 向持有 Alice 隐私信息的目标数据控制者发送虚假的删除通知请求。这导致数据控制者在未经授权的情况下删除了 Alice 的个人隐私信息, 不仅影响了 Alice 与数据控制者的正常服务, 还严重扰乱了数据控制者的正常运营。
3. **路径重定向:** 路径重定向发生在恶意攻击者 Eve 通过更改目标地址, 将删除通知重定向到一个与原通知无关的数据控制者的情况下。这种攻击行为导致不相干的数据控制者错误地删除用户隐私数据, 而预期的接收者未能接收和处理通知, 导致数据无法删除。例如, Eve 截获了一个从数据控制者 M 发到数据控制者 N 的 Alice 的删除通知请求, 将其重定向至数据控制者 P。最终导致数据控制者 P 删除了 Alice 的隐私数据, 而预期的接收者 N 没有处理该通知请求, 从而影响了用户 Alice 在数据控制者 N 处的正常服务, 为用户和数据控制者都带来了重大风险。
4. **重放攻击:** 重放攻击是指恶意攻击者 Eve 截获并保存了先前接收到的合法删除通知, 并在一段时间后再次发送被截获的删除指令, 以欺骗数据控制者试图模拟合法用户的删除请求。例如, 恶意攻击者 Eve 在某个时刻截获并保存了某数据控制者转发给目标数据控制者的 Alice 的删除请求。在未来的某个

时刻, Eve 将其重新发送给目标数据控制者, 仿佛是 Alice 重新提交了通知。然而, 在 Eve 重放该请求之前, Alice 已经上传了新的数据。目标数据控制者错误地将重放的通知视为合法请求, 导致 Alice 新上传的信息被错误地删除。

5. **丢弃攻击:** 丢弃攻击是指恶意攻击者 Eve 截获了从某数据控制者转送至目标数据控制者的删除通知请求, 并将其丢弃, 同时向上一跳数据控制者发送伪造的确认消息。这种欺骗旨在迫使目标数据控制者违反删除权合规规定。例如, Eve 截获并丢弃了一个从数据控制者 M 到数据控制者 N 用以删除 Alice 数据的通知, 然后向数据控制者 M 发送伪造的删除成功消息。最终, 数据控制者 M 不知道删除并未执行, 从而面临相应的处罚, 而 Alice 的隐私数据则未按照其意愿得到删除。

本毕设假设恶意攻击者无法观察或修改运行在可信执行环境内的任何数据或程序代码, 并且可信执行环境本身被信任, 其中运行的代码得到了对应的完整性保证。尽管现有的可信执行环境解决方案(例如本毕设所基于的 SGX)并不能完全满足上述抽象的安全假设。针对可信执行环境的改进措施是一个独立的研究领域, 可信执行环境解决方案的安全问题超出了本毕设的讨论范围, 目前仍在积极进行研究与改进。<sup>[25-26]</sup>。

## 2.3 理论技术

### 2.3.1 HMAC

HMAC (Hash-based Message Authentication Code) 是一种基于散列函数的消息认证码机制, 旨在验证消息的完整性与真实性。其核心思想是将消息与密钥结合, 通过散列函数的迭代计算, 生成一个固定长度的认证码, 以验证消息在传输过程中是否被篡改或伪造。

HMAC 现已成为事实上的 Internet 安全标准, 是当前众多安全协议中普遍采用的认证服务提供方式(例如 TLS/SSL、IPsec 与 SSH, 以及身份验证和授权机制中的 OAuth 和 OpenID)。基于散列函数的消息认证码机制能够与任何迭代散列函数兼容使用, 由于密码散列函数的库代码来源较为广泛, 这也为开发人员提供了更多的选择与便利。相较于基于分组密码的消息认证码机制, 该机制执行速度快, 在密钥的使用与处理方面也相对简单。若已知嵌入的散列函数强度, 则可以完全推断出认证机制抵抗密码分析的强度。

在本次毕设中，删除指令可信通知与转发协议中的标签 Auth 生成算法基于 HMAC 进行实现，用于生成认证标签，以便于对输入数据进行验证。具体的标签生成算法会在章节 3.2.1 中进行详细描述，在此不做过多阐述。

### 2.3.2 Mininet 网络仿真器

Mininet 是一款用于创建虚拟网络环境的开源网络仿真工具。旨在提供一个轻量级、灵活可定制且易于使用的虚拟网络实验平台。该工具基于 Linux 操作系统，利用 Linux 内核功能以及虚拟化技术模拟网络拓扑，广泛用于验证网络算法、协议设计、SDN（软件定义网络）应用开发等方面。

Mininet 允许用户在单个物理主机上创建一个或多个虚拟网络，并在这些虚拟网络中模拟主机、交换机、路由器和链路等网络设备。每个虚拟设备都运行在独立的网络命名空间中，具有独立的网络栈和 IP 地址。同时，Mininet 提供了一个高度可控的定制化环境，用户可以通过编程方式定义和配置网络拓扑，设置设备的属性和行为，并生成各种网络流量来进行网络仿真实验。

此外，Mininet 在 SDN（软件定义网络）开发中也起到关键作用，SDN 是一种网络架构范式，通过将网络的控制平面与数据平面分离，实现了网络的灵活性与可编程性。在 SDN 中，网络设备（如交换机、路由器）的控制逻辑由集中式的控制器进行管理与控制，而数据流则通过 OpenFlow 协议与控制器进行交互。通过 Mininet 创建虚拟的 OpenFlow 交换机，并与 OpenFlow 控制器进行交互，模拟和测试 SDN 网络的行为，用户能够快速构建、测试和优化 SDN 网络。

在本次毕设中，为评估在真实网络环境下网络功能模块（部署于业务平面）所需的资源开销，本毕设设计了一系列仿真模拟实验。相关实验借助 Mininet 网络仿真器，利用 Chinanet 等网络拓扑得以实现，详见章节 5.2.1。

### 2.3.3 可信执行环境 (TEE)

可信执行环境（Trusted Execution Environment, TEE）是一种安全的计算环境，旨在保护敏感数据和执行关键任务。它提供了一种隔离的执行环境，在该环境中，应用程序和数据可以受到保护，不受操作系统或其他应用程序的干扰或恶意攻击。

其中，Intel SGX 是当前领先的可信执行环境（TEE）技术之一，其核心通过

扩展指令集与内存访问控制来创建一个被称为 **Enclave** 的安全容器。这个容器在应用程序的地址空间内部划分出一块受保护区域，旨在为容器内的代码和数据提供密封的保密性和完整性保护。

这种隔离机制的实现依赖于 **SGX** 程序执行过程中 **CPU** 状态的切换。当程序请求进入飞地时，**CPU** 首先会验证请求的合法性，随后切换状态为受保护的模式。在此模式下，内存加密引擎 (**Memory Encryption Engine, MEE**) 会对飞地内外的数据流进行加解密，确保隐私数据在物理层面的安全。此外，**CPU** 还会通过 **Enclave Page Cache Map (EPCM)** 管理内存页面的访问权限，防止飞地遭到未授权的操作。

在本次毕设中，涉及的信任分发验证机制是在 **Intel SGX** 上实现的，详见章节 5.2.1。

#### 2.3.4 JMeter 性能测试工具

**JMeter** 是一款基于 **Java** 的开源性能测试工具，被广泛用于对软件、Web 应用程序、API 以及服务器进行负载测试、性能测试与压力测试。它由 **Apache** 软件基金会开发和维护，并且提供了丰富的功能与灵活的配置选项。

**JMeter** 支持多种协议，包括 **HTTP**、**HTTPS**、**FTP**、**SOAP**、**REST**、**JDBC** 等。此外，其支持测试计划与分布式测试，允许用户通过添加线程组、定时器、取样器、断言器等元素来构建复杂的测试场景。同时能够将测试负载分布到多个 **JMeter** 实例上，以模拟更大规模的用户并发访问。最后，**JMeter** 提供了实时监控与性能数据收集功能，包括响应时间、吞吐量以及错误率等指标。支持生成测试报告，用于可视化测试结果和性能指标，以便进行分析与比较。

本次毕设使用了 **JMeter** 作为性能测试工具，对删除通知可信分发系统的性能进行了详尽测试。详见章节 5.3.2。

## 第三章 方案设计

本章主要介绍了删除通知可信分发系统的总体方案设计及其功能，并详细描述了其构成子系统的设计和作用。最后介绍了删除通知可信分发通信协议，并逐一介绍了协议所用机制的原理细节。

### 3.1 删除通知可信分发系统设计

#### 3.1.1 系统总体设计

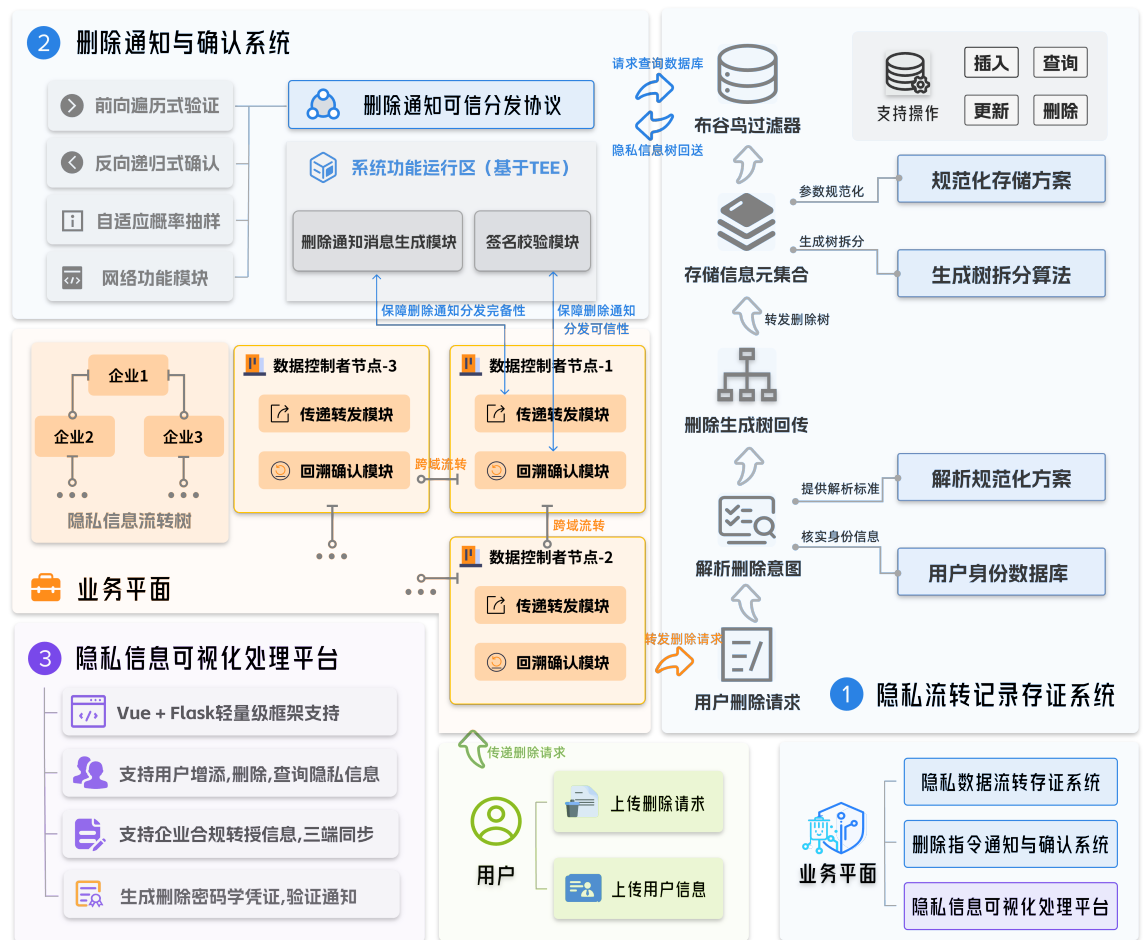


图 3.1 系统整体框架

本毕设设计并开发了一套删除通知可信分发系统，整体框架参见图 3.1，该系统基于跨域删除通知可信分发协议得以实现，致力于为删除权网络基础设施提供技术支持。在生态架构层面，系统主体基于一种跨层协同的中心化管理架构进行开发，以保障删除通知在数据控制者之间安全可信地进行分发。该管理架构由业务平面和监管平面构成。监管平面主要由中心监管机构构成，其职责是记录各数

据控制者之间的数据共享传播历史。业务平面由广大的数据控制者构成，除了正常的业务活动，当数据控制者需要和其他数据控制者互相跨域共享数据的时候，应向监管平面进行存证。在通信协议层面，本毕设计并定义了一套跨域删除通知可信分发协议，以辅助相关数据控制者安全地分发删除通知，当需要保障用户删除权时，业务平面通过该协议向监管平面提出申请，以获得该用户流转树的下一跳数据控制者，并依次按照流转树转发删除指令。

在代码实现层面，该系统基于跨域删除通知可信分发协议得以实现，部署于跨层协同的中心化管理架构之下，借由删除通知可信协议进行平面间通讯，主要由三大子系统构成：

1. **删除指令通知与确认系统：**该子系统部署于业务平面与监管平面之上，承载着多项关键任务，例如平面间通信交互，删除通知生成，删除指令通知与确认，删除指令验证等。该系统借助删除通知可信协议，使得数据控制者能够与监管机构进行安全可信的信息共享。确保指令的执行能够得到及时反馈，为数据控制者有效履行删除权提供技术保障。
2. **隐私数据流转状态管理与存证系统：**该子系统部署于监管平面上的中心监管机构，负责存储用户隐私信息的流转情况，并为删除指令通知与确认系统构建删除通知提供相应决策依据。依托其存储的相关隐私流转信息，可以高效响应生成并下发对应的删除通知树，便捷地验证可信删除凭证的正确性，为删除指令的通知与确认工作提供安全保障。
3. **隐私信息处理可视化平台：**为了提升系统整体的易用性与便捷性，本毕设使用 Vue 框架开发了隐私信息处理可视化平台。针对参与删除通知与确认任务的用户、中心监管机构端与数据控制者三方，本毕设分别设计了用户端、中心监管机构端和数据控制者端操作平台。该平台具备诸如用户端授权隐私，用户端提交删除请求，数据控制者端转发删除请求，数据控制者端处理删除通知，中心监管机构端监督验证删除通知过程等功能。三端互联，直观效果强，且交互性良好。

## 3.1.2 系统功能

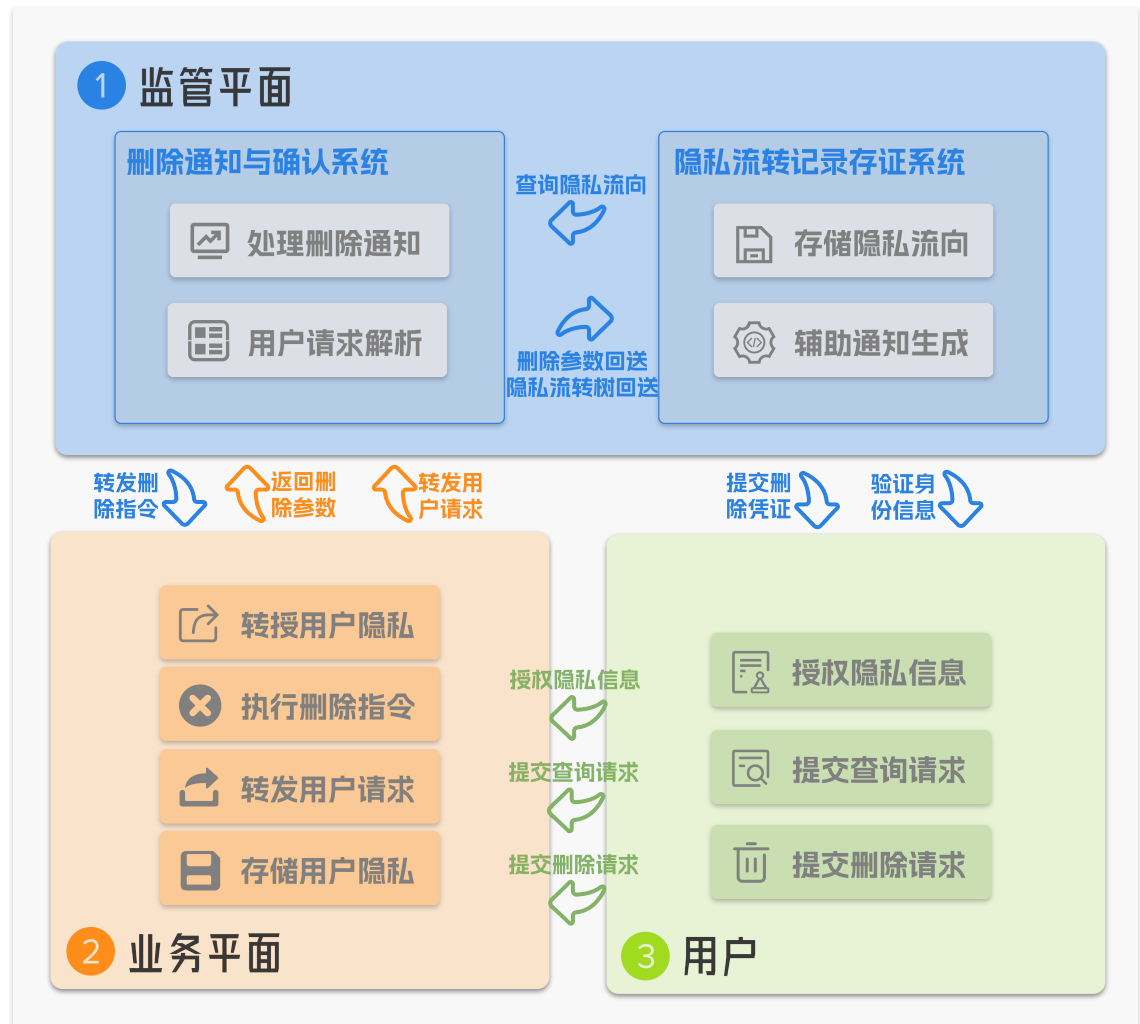


图 3.2 系统主要功能

本毕设实现的删除通知分发系统，参考真实删除情景与删除通知分发的基本流程（参考章节 4.1）。考虑设置用户，中心监管机构与数据控制者三端，结合各方需求进行系统功能开发，其主要功能参见图 3.2。

在用户端，主要功能包括设置授权隐私信息、提交查询请求以及提交删除请求。用户可以设置和管理自己的隐私信息，并向系统提交查询请求以获取相关数据。还能够发起隐私信息删除请求，将该请求提交给业务平面内部署的数据控制者进行处理。在业务平面内，数据控制者扮演重要角色，其主要功能包括转发用户请求、存储用户隐私、执行删除指令以及转授用户隐私等操作。在监管平面，监管机构由删除通知与确认系统以及隐私流转记录存证系统构成，负责实现本系统的核心功能：在不可信数据控制者网络之间分发删除通知消息。

### 3.2 删除通知可信分发协议

个人隐私数据在不同数据控制者（即不同的互联网平台，例如：腾讯、百度等）之间进行着跨域共享流转。其传播途径在拓扑上构成了一个以数据管理者为节点，删除通知数据包在不可信网络中的跨域流转路径为边的传播树。因为要通过数据控制者之间的不可信网络，面临着诸如篡改、丢弃、重放等一系列安全问题。这会导致实际删除通知的过程和预期不一样。源企业节点，作为用户履行删除权的代表，需要知晓整个删除通知流转过程是否按照获取的流转树执行。对此，本毕设需要在业务平面设计一种安全分发协议，保证删除通知按照预期分发，并允许源数据控制者知晓分发结果。

本毕设将删除通知的分发与确认过程，拆分成两个子过程：**删除通知的前向分发过程以及删除确认的反向回溯过程**。前者是指删除通知由源企业节点逐层向删除通知树上叶子节点分发的过程，用以通知各相关数据控制者执行适当的删除操作；而后者是由各叶子节点沿着删除通知树朝根节点逐级确认的过程，帮助源数据控制者确认各数据控制者是否收到删除通知。考虑到数据流转历史的随机性与不确定性，同时为了减少系统开销，本毕设基于 HMAC 设计了一种逐跳验证的删除通知正向分发验证协议，用以保证删除通知在每一对数据控制者之间都被正确地转发，同时可以应对各种形式的数据流转。

该协议定义了一套标准化流程，涵盖了删除通知分发与确认的全过程。旨在为删除通知可信分发系统提供流程化的实现依据，以确保整个系统在处理删除通知时遵循一致的规范和流程，进而实现可信分发和确认的目标。

#### 3.2.1 前向遍历式验证机制

跨域删除通知可信分发协议定义了一套标准化流程，用于处理和转发删除通知，确保所有相关数据控制者都能收到删除请求。通过协议头部字段，使得删除请求的来源、目的地以及必要的操作指示能够清晰地标识和传达，从而实现对数据删除通知的精确控制和监督。核心协议头部的设计包含用户唯一标识符（ $U_{ID}$ ）、通知类型（Type）、时间戳（T）、首个数据控制者 ID（ $C_{ID}^{first}$ ）、目标数据控制者 ID（ $C_{ID}^{des}$ ）、操作请求（Req）以及认证字段（Auth）（参考图 3.3）。

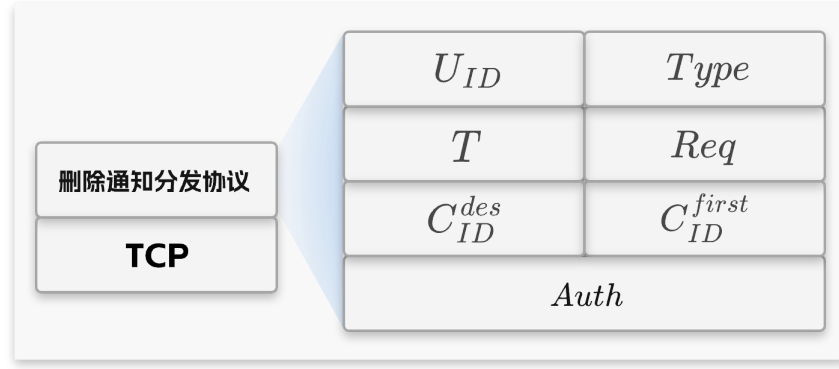


图 3.3 删除通知可信分发协议头

协议头部字段设计如下:

- $U_{ID}$ : 代表删除通知过程中用户的唯一标识符
- $Type$ : 用以区分删除通知的来源方,  $Type = 0$  时表示此删除通知由用户发出,  $Type = 1$  时表示该删除通知由数据控制者转发,  $Type = 2$  时表示删除通知的 ACK 确认消息。
- $T$ : 表示时间, 数据控制者在接收到删除通知后进行计算:  $\Delta t = t_2 - t_1$ 。  $C_{ID}^{first}$  表示删除通知的前一个数据控制者, 当  $Type$  的值为 0 时, 该字段的信息无意义。  $C_{ID}^{des}$  表示接收删除通知的下一个数据控制者, 当  $Type$  的值为 2 时, 该字段的信息无意义。
- $Req$ : “ $Type$ ” 的值并不承载数据传输的实际含义, 而是用于识别删除过程中的各种操作, 例如删除所有用户个人信息、年龄信息或身份证号码等, 即不同类型的删除活动。具体的映射关系和可能的扩展功能应根据相关互联网数据控制者的需求和标准进行定义和实现。
- $Auth$ : 由一个信息四元组与会话密钥  $k$  组成, 如下所示:

$$Auth \leftarrow HMAC((U_{ID} \parallel T \parallel Type \parallel C_{ID}^{des}), key) \quad (3-1)$$

在跨域删除通知可信分发协议中, 采用逐跳验证方式来确保删除通知的安全传输。每当删除通知被转发给下一个数据控制者时, 接收方数据控制者会用自己的 ID 替换协议头中的目标数据控制者 ID ( $C_{ID}^{des}$ ), 然后根据更新后的信息重新计算认证字段  $Auth'$ :

$$\text{Auth}' \leftarrow \text{HMAC}((U_{ID} \parallel T \parallel \text{Type} \parallel C_{ID}^{\text{own}}), \text{key}) \quad (3-2)$$

这样每一跳的认证都基于当前接收者的独有信息，加强了消息传递过程的安全性。通过这种逐跳验证的认证方式，删除通知分发协议能够有效地防止数据篡改和伪造，确保删除通知在各数据控制者之间安全、可靠地传递。

### 3.2.2 反向递归式确认机制

本方案提出一套了反向递归式确认机制，用于管理删除指令的确认过程。首先，每个数据控制者在接收到删除指令后，会进入**安全验证状态**。在此状态下，数据控制者对相关数据包进行安全性验证，以检查删除通知消息是否遭到攻击（例如篡改或伪造）。如果验证失败，即发现删除通知消息遭到攻击，数据控制者立即向其上一级数据控制者报告该情况，同时进入**预警状态**，并层层向上传递攻击警告，直至源数据控制者，并终止转发删除指令。

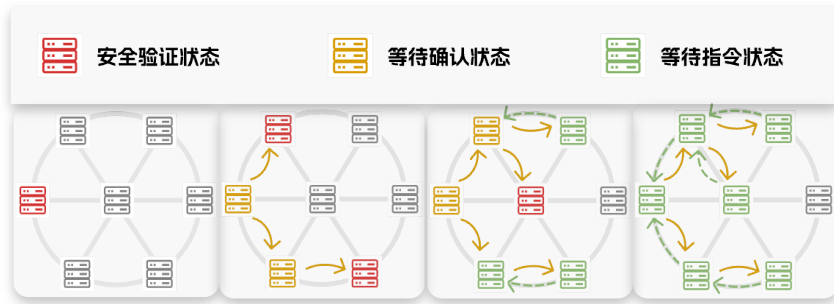


图 3.4 反向递归式确认机制

如果验证成功，数据控制者则使用计数器跟踪待接收确认的子节点数量，随后进入**等待确认状态**。在等待确认状态中，数据控制者等待所有下级节点的确认信息，一旦数据控制者收到其所有下级节点的确认，它将向其上一级数据控制者节点返回确认信息。最后进入**等待指令状态**，这一状态表示该数据控制者所有下级节点都已成功处理删除指令，并且整个分支准备好继续进行正常操作。详见图 3.4 与算法 3.1。

---

 算法 3.1 Reverse Recursive Confirmation Algorithm
 

---

```

1: Input: Root node of a data controller network
2: Output: Network processed following RRC
3: procedure RRC(root)
4:    $Q \leftarrow$  empty queue
5:    $R \leftarrow$  empty list
6:   Enqueue (root, 'INIT') into  $Q$ 
7:   while not  $Q.isEmpty()$  do
8:     (node, phase)  $\leftarrow Q.dequeue()$ 
9:     if phase = 'INIT' then
10:       Enqueue (node, 'Secure Verification State') into  $Q$ 
11:     else if phase = 'Secure Verification State' then
12:       if security check passes then
13:         Enqueue (node, 'Awaiting Confirmation State') into  $Q$ 
14:       else
15:         Enqueue (node, 'Warning State') and its ancestors into  $Q$ 
16:       end if
17:     else if phase = 'Awaiting Confirmation State' then
18:       if all children confirmed or no children then
19:         Enqueue (node, 'Awaiting Instruction State') into  $Q$ 
20:          $R.append(node)$ 
21:       else
22:         Wait for all children to confirm
23:          $R.extend(children\_acknowledgements)$ 
24:       end if
25:     else if phase = 'Awaiting Instruction State' then
26:       Notify parent, if any, and adjust parent's state accordingly, transmit  $R$  to parent
27:     else if phase = 'Warning State' then
28:       Process security alert, notify all relevant parties
29:     end if
30:   end while
31: end procedure

```

---

### 3.2.3 自适应概率抽样验证

本毕设开发了一种自适应概率抽样验证方法，旨在在保持低系统开销的同时实现准确的违规检测。对于给定的时间间隔，当客户端发送删除请求的强度较低时，以较高的抽样概率验证已发送的删除通知；而当大量客户端要求执行其对应的删除权时，以相对较低的概率进行验证。

这种方法通过允许系统根据感知到的删除通知强度调整抽样概率，实现了自适应的风险管理。例如，在接收到较少删除通知的情况下，应增加抽样概率以增强安全措施，因为高抽样率可以及时检测和应对恶意攻击。相反，当观察到高强度的删除通知时，系统可以降低抽样率以节约资源，从而优化安全和运营效率之间的平衡。

具体操作过程中，每个数据控制者首先使用会话密钥  $k$  对 Auth 字段执行键控伪随机函数 PRF，然后将结果与预定义的采样概率进行比较：

$$\text{PRF}_k(\text{Auth}) \leq P_s \quad (3-3)$$

若满足上述条件，数据控制者将验证 Auth 字段并发送确认通知。此外，每个数据控制者维护一个计数器 CountS，用于记录已处理的删除通知的数量。采样概率  $P_s$  通过 CountS 与预定义的常量  $u$  动态计算。随着 CountS 的增加，采样概率相应地减小；当 CountS 较小时，采样概率增加，允许验证更多通知。计算方法如下：

$$P_s(\text{CountS}) = \frac{1}{(1+u)^{\text{CountS}}} \quad (3-4)$$

然而，直接应用此采样公式可能会导致在处理大量删除通知后采样概率过低。对此，本方案在采样公式中引入了一个最小采样概率  $P_{\min}$ ，以防止采样概率过低。不同的数据控制者可根据自身应用场景与需求调整相关参数，以达到最佳的效率和成本平衡。因此，最终的采样概率计算公式如下所示：

$$P_s(\text{CountS}) = \begin{cases} \frac{1}{(1+u)^{\text{CountS}}}, & \text{if } \frac{1}{(1+u)^{\text{CountS}}} > P_{\min} \\ P_{\min}, & \text{if } \frac{1}{(1+u)^{\text{CountS}}} \leq P_{\min} \end{cases} \quad (3-5)$$

具体参数配置应由数据控制者依据实际需求灵活调整。在对安全性的要求较

高时可以适当提升采样率以增强系统的安全性能。此外，数据控制者还可以根据数据量、处理速度和系统负载等多种因素，综合决定最合适的参数设置。

#### 算法 3.2 自适应验证机制

**Require:**  $Auth, k, CountS, P_{\min}, u$

**Ensure:** Verification decision based on probabilistic sampling

```

1: function SAMPLING( $Auth, k, CountS, P_{\min}, u$ )
2:    $P_S \leftarrow \max\left(\frac{1}{(1+u)^{CountS}}, P_{\min}\right)$ 
3:    $Sample \leftarrow \text{PRF}_k(Auth)$  ▷ Perform PRF on Auth
4:   if  $Sample \leq P_S$  then
5:     Verify  $Auth$ 
6:     return True
7:   else
8:     return False ▷ Skip then forward directly
9:   end if
10: end function

```

#### 3.2.4 删除通知分发网络功能模块

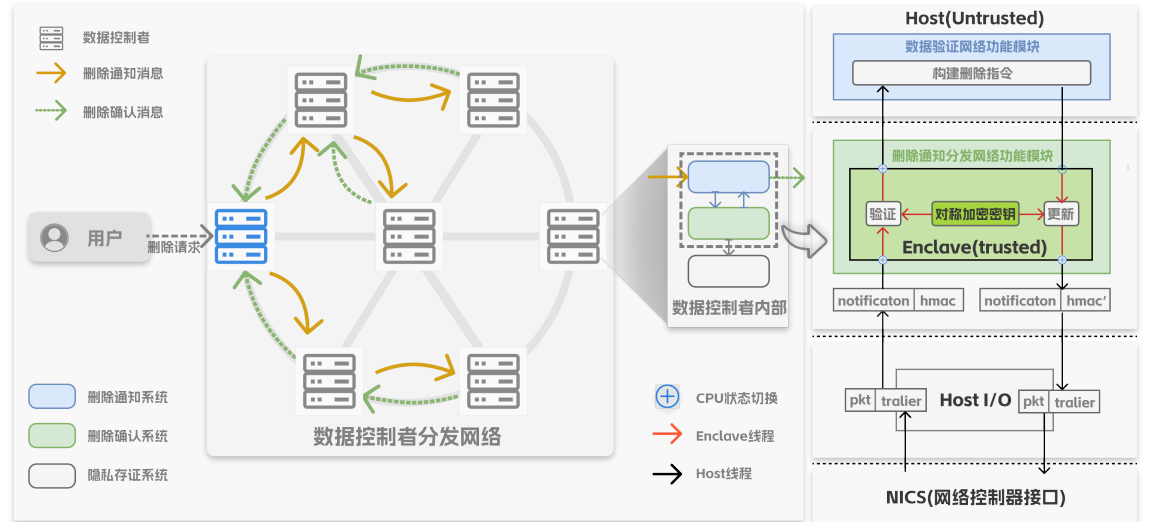


图 3.5 删除通知分发网络功能模块

本毕设涉及的信任分发验证机制是在 Intel SGX 上实现的 (详见图 3.5)，在本方案中，指令的核验过程将在 Enclave 内进行，设计的一个关键特性是在 SGX 飞地中存储用于验证请求的密钥，并在其中执行验证过程。这一过程主要涉及对用户以及数据控制者所发送的删除通知进行密钥验证，确保每个指令都是合法且经过授权的。由于飞地具有的隔离性质，攻击者以及不诚实的数据控制者无法直接

访问或修改飞地中的代码和数据，这也就意味着其无法伪造或者篡改验证结果。这种独立的验证机制为数据的删除和处理流程提供了额外的安全层，这大大增强了对删除指令流过程的监管能力，确保了验证机制的安全性和可靠性。

为了给数据控制者提供一种“即插即用”的轻量级解决方案，本毕设将上述协议通过 **network function** 的形式实现。该 NF 不会对数据控制者的正常业务造成影响，主要负责在数据控制者之间分发删除通知，同时通知数据控制者自己负责的管理域中负责数据删除的系统，以便开展删除操作。本毕设利用可信执行环境（TEE）技术（例如 Intel 的 SGX 和 ARM 的 TrustZone），来在主机内部建立一个安全的隔离区域，用于执行安全分发删除通知的网络功能。关键的安全相关操作被限制在隔离区域内进行，而与删除通知过程相关的其他任务，如消息体解析和确定下一跳数据控制者，则在标准内存空间中得到运行。这种架构不仅增强了该协议的安全性，还最大限度地减少了在正常执行状态和隔离执行状态之间切换的开销。此外，通过可信执行环境，参与删除通知分发过程的所有数据控制者可以使用相同的对称加密密钥，并在安全可信的环境中执行相关的加密操作。

### 3.3 协议流程

#### 3.3.1 系统模型

该协议旨在展现跨层协同中心化架构下隐私信息初始化以及删除通知可信分发的全过程。协议部署在由三方参与者协作构建的隐私信息流转系统模型下，主要包括以下实体：中心监管机构（RA, Regulatory Agency），企业节点网络（BN, Business Node）以及用户终端（UE, User Terminal）。

#### 3.3.2 隐私信息初始化流程

为模拟在真实场景下用户对隐私信息流转历史的注册行为，同时初始化企业节点及中心监管机构相关存储数据结构。本协议方案设置了隐私信息初始化流程，由用户选择提交个人隐私信息流转历史，企业接收请求并建立存证数据表进行存储，最后交由监管机构进行进一步登记。监管机构负责存储所有下辖企业节点贮藏的隐私信息流转历史记录。具体过程如图 3.6 所示。

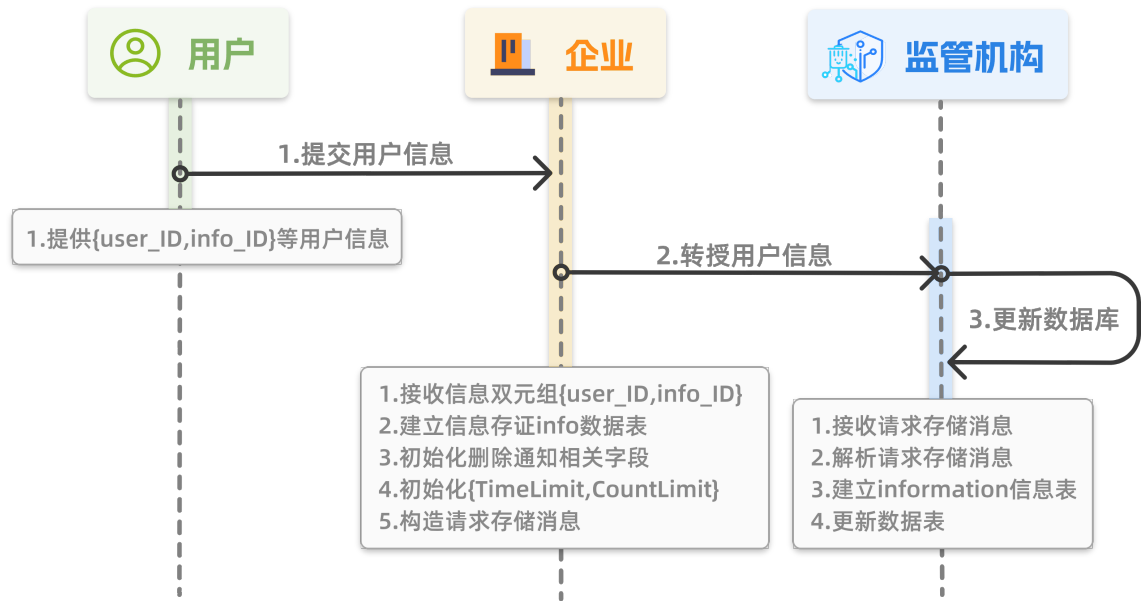


图 3.6 用户隐私信息初始化流程图

用户隐私信息初始化的流程如下：

**STEP1:** 用户将信息二元组 (user\_ID,info\_ID) 提交至对应企业节点，代表用户将相应个人隐私信息流转历史授权给相应企业，此处 user\_ID 表示用户唯一标识符，info\_ID 代指转授隐私信息类型。

**STEP2:** 企业节点接收来自用户的信息二元组，同时判断是否已为该用户建立对应信息存证 info 数据表。若该用户为新用户，则为其建立对应 info 数据表。

**STEP3:** 企业节点初始化 TimeLimit, CountLimit 字段表。CountLimit 字段负责实现计次触发功能。

**STEP4:** 企业节点初始化删除通知相关字段，涵盖 DeleteMethod, DeleteGranularity, DeleteNotifyTree, DelConfirmSignatureDict 以及 triggerType 等字段，旨在提供细粒度的删除选项。

**STEP5:** 企业节点构造并向中心监管机构发送请求存储消息，消息体中涵盖了信息二元组，源企业节点 ID 以及前述已初始化字段。

**STEP6:** 中心监管机构接收并解析来自于企业节点的请求存储消息。同时在隐私信息历史存证系统中更新（或建立）该企业节点所对应数据表。中央监管机构作为事实上的信息流转中心，用于存储用户隐私流转记录，同时记录了相应删除信息，以进一步支持计次触发删除功能。

3.3.3 删除通知可信分发流程

本小节旨在介绍删除通知在不可信数据控制者网络之间进行分发的基本流程，并进一步展示协议的原理机制与部分实现细节。在遵守可信分发协议的前提下，由用户选择提交个人删除请求，中心监管机构作为后台型的管理机构，与企业协同合作，共同保障删除通知分发的完备性与可信性。

方案同时于小节 3.3.3.1，小节 3.3.3.2以及小节 3.3.3.3定义了三种删除通知可信分发核心算法，分别为删除通知消息生成算法，删除通知消息解析签名算法以及删除反馈消息生成算法。具体过程如图 3.7所示。

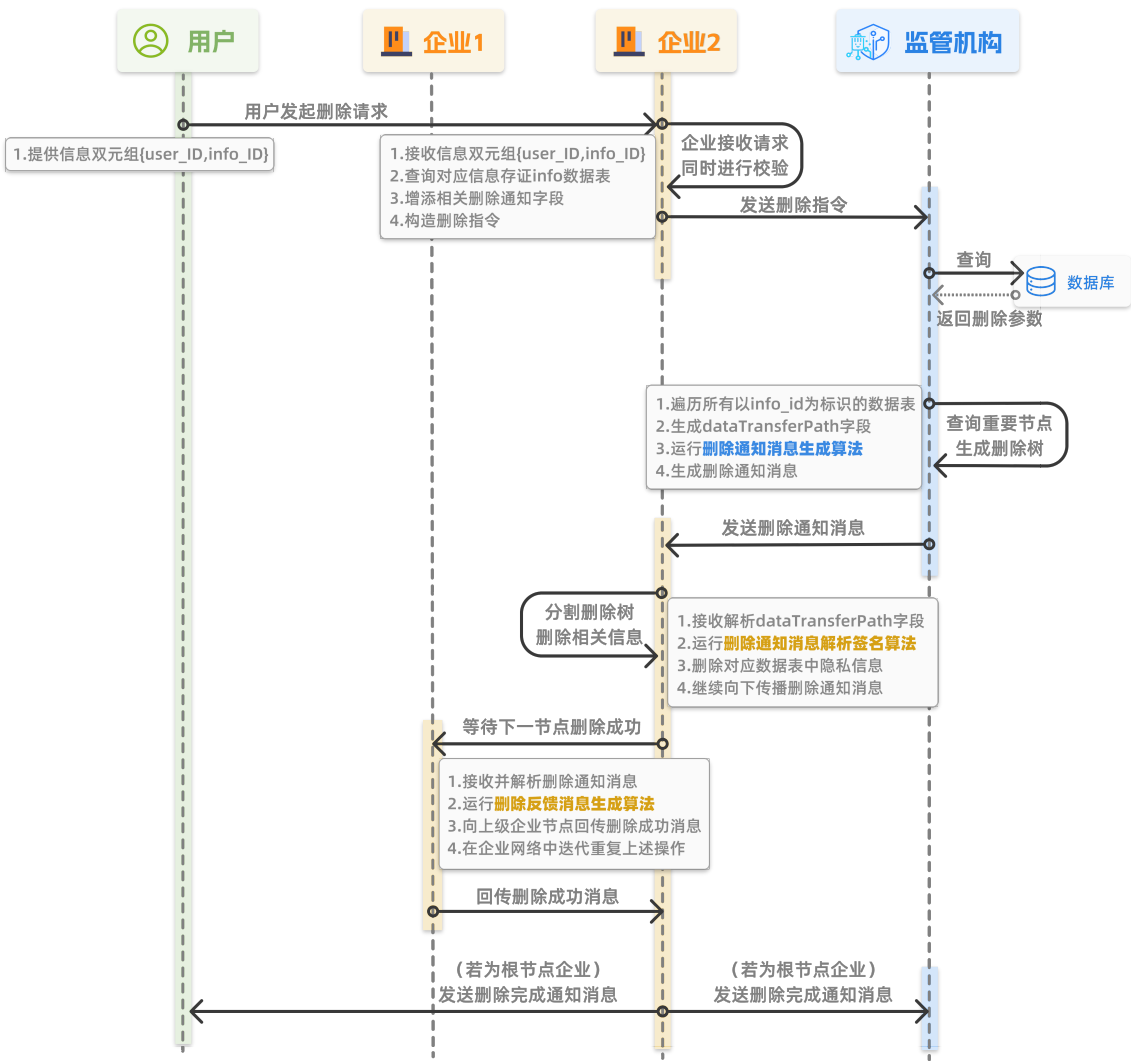


图 3.7 删除通知可信分发流程图

删除通知可信分发流程如下：

**STEP1:** 用户将信息二元组 (user\_ID,info\_ID) 提交至对应源企业节点（用户首次授权节点），同时向该企业正式发起删除请求。

**STEP2:** 企业节点接收并解析来自于用户的删除通知请求，同时查询企业节点内置的对应 info 数据表，校验用户身份，增添相关删除通知字段，使用表中的参数构建删除指令并通知中心监管机构。

**STEP3:** 中心监管机构接收来自源企业节点的删除通知消息，通过遍历所有以 info\_id 为标识的数据表，生成对应删除传播通知树（即 dataTransferPath 字段，数据类型为 Json），使用数据库中返回的参数运行**删除通知消息生成算法**，构造并回送删除通知消息。

**STEP4:** 企业节点接收并解析来自中心监管机构的删除通知消息，提取 dataTransferPath 字段，并运行**删除通知消息解析签名算法**，以获取（同时删除）对应企业内置数据表中的隐私信息。随后源企业节点在删除通知消息上留下签名，并按照删除传播通知树所规定的路径向下一级企业节点继续传递该消息。

**STEP5:** 下一级企业节点接收并解析来自于上一级企业节点的删除通知消息，同时运行**删除反馈消息生成算法**，向上级企业节点回传删除反馈消息。

**STEP6:** 相关企业节点不断迭代重复 STEP4 与 STEP5，直至删除传播通知树上的所有企业节点均已完成签名，此时完成签名后的删除通知消息将被逐级回送至源企业节点处，这表明用户的隐私信息得到了完备的跨域删除。

**STEP7:** 源企业节点向监管机构提供签名后的删除通知消息，以供中心监管机构进一步核实处理。同时告知用户删除请求已得到落实。

### 3.3.3.1 删除通知消息生成算法

中心监管机构在生成对应删除传播通知树后，将使用删除通知消息生成算法构造删除通知消息。其算法运行图如图 3.9 所示。



图 3.8 删除传播树数据初始化

**删除通知消息生成算法步骤如下：**

**STEP1:** 中心监管机构首先对删除传播通知树进行初始化处理。首先创建 SL 与 Segment\_tree 结构树（仅具有单个根节点，且根节点 ID 记为 root），并将 SL 与

Segment\_tree 分别添至删除传播通知树末端,使之成为删除传播通知树的新叶子节点,并将其 ID 记为 leave\_i。

**STEP2:** 中心监管机构对删除传播通知树进行层序遍历,若当前遍历节点为叶子节点,则不执行任何操作。若当前遍历节点不为叶子节点,则使用当前遍历的企业节点所对应的 secret\_key 计算 session\_key:

$$\text{session\_key} = \text{Hash}(\text{secret\_key} + \text{time\_stamp}) \quad (3-6)$$

其中, secret\_key 为当前企业节点所持有的密钥, Hash(.) 表示哈希运算, “+” 表示拼接, time\_stamp 为时间戳, 此处时间戳有两个主要作用, 其一是帮助删除传播树中各个节点计算会话密钥, 其次是防止恶意攻击者进行重放攻击。

**STEP3:** 紧接着, 中心监管机构对删除通知树进行遍历。若遍历到的企业节点为源企业节点, 则中心监管机构计算其  $\text{pktMAC} = \text{HMAC}(\text{session\_key}, \text{payload})$ ; 若遍历到的企业节点为非源企业节点且非删除传播树中新增的叶子节点, 则中心监管机构计算其  $\text{pktMAC} = \text{HMAC}(\text{session\_key}, \text{payload} + \text{SL}_{\text{position}})$ 。其中  $\text{SL}_{\text{position}}$  为与当前遍历到节点位置所对应的删除确认树所存储的 SMAC。

**STEP4:** 接下来, 对于删除通知树中的每个节点, 中心监管机构使用该节点的 index 计算 tmp:

$$\text{tmp} = \text{parent\_id}(\text{or center}) + \text{pktMAC} + \text{time\_stamp} + \text{index} \quad (3-7)$$

若遍历到的企业节点为源企业节点时, 上述计算公式需使用 center 字段参与运算, center 为中心监管机构的标识。若遍历到的企业节点为非源企业节点, 上述计算公式则使用 parent\_id 字段参与运算, parent\_id 表示父节点 ID。随后中心监管机构使用 tmp 与该节点的 ID 计算 SID:

$$\text{SID} = \text{xor}(\text{child\_id}, \text{HMAC}(\text{session\_key}, \text{tmp})) \quad (3-8)$$

若遍历到的企业节点的子节点为删除传播树新增加的叶子节点时, 则该企业的 child\_id = center。最后中心监管机构使用 SID 计算 SMAC:

$$\text{SMAC} = \text{HMAC}(\text{session\_key}, \text{SID}) \quad (3-9)$$

其中, `payload` 为删除通知消息信息载荷,  $HMAC(\cdot)$  表示计算哈希消息验证码的算法,  $xor$  表示异或运算; `index` 为子节点编号, `child_id` 表示子节点 ID, `SID` 表示跨域流转信息, `SMAC` 表示依据 `SID` 进行对应计算得出的认证码。

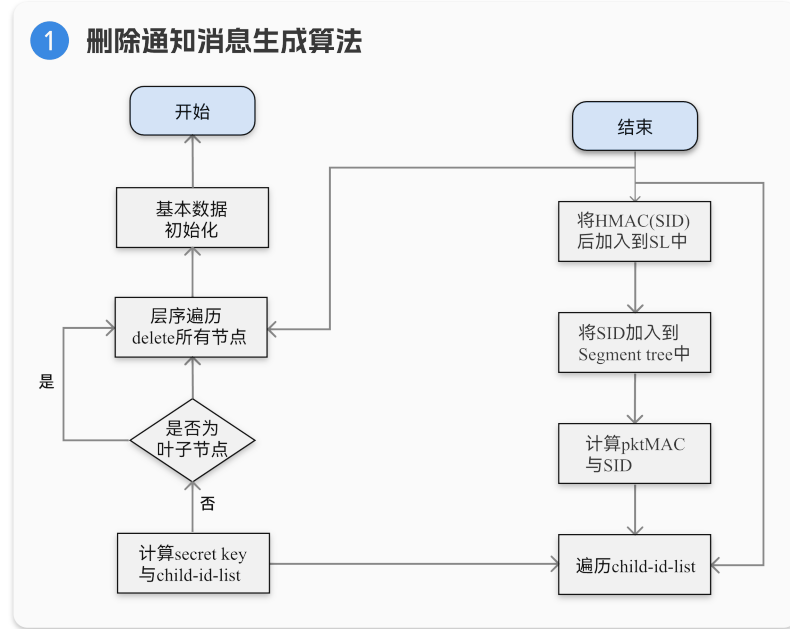


图 3.9 删除通知消息生成算法运行图

### 3.3.3.2 删除通知消息解析签名算法

企业节点在接收到来自中心监管机构的删除通知消息后, 对删除通知消息进行解析与签名, 根据 `dataTransferPath` 字段内涵盖的隐私信息跨域流转历史, 对其父节点进行验证, 并进一步确定其子节点, 同时向下一级企业节点传递删除通知消息。其算法运行图如图 3.9所示。

**删除通知消息解析签名算法步骤具体如下:**

**STEP1:** 当前企业节点从接收到的删除通知消息中提取删除传播树 `dataTransferPath` 字段, 时间戳 `time_stamp` 以及信息载荷 `payload`, 并获取该删除传播树中的根节点与其余各个节点所存储的 `SID`, 同时计算 `session_key`:

$$\text{session\_key} = \text{Hash}(\text{secret\_key} + \text{time\_stamp}) \quad (3-10)$$

**STEP2:** 若当前企业节点为源企业节点, 则企业直接计算得到 `pktMAC`。若当前企业节点非源企业节点, 则当前企业节点利用接收到的删除通知树 `dataTransferPath` 字段中所存储的 `SID` 来计算 `SMAC`, 并判断该 `SMAC` 是否与该删除通知树中所存储的 `SID` 相等; 若相等, 则当前企业节点对来自其父节点的删除通知消息验证通

过，否则验证不通过。父节点验证通过后，当前企业节点将该删除通知树中所存储的  $SID$  作为  $SL_{\text{position}}$  进而计算得到  $pktMAC$ 。

**STEP3:** 接下来，面向该企业节点的其余各个子节点，使用子节点的  $index$  计算  $tmp$ :

$$tmp = center + pktMAC + time\_stamp + index \quad (3-11)$$

同时，使用其余各个子节点的  $SID$  计算出  $child\_id$ ，以得到当前企业的子企业节点 ID:

$$child\_id = xor(business_{id}, HMAC(session\_key, tmp)) \quad (3-12)$$

**STEP4:** 随后，面向其余各个子节点，当前企业节点从收到的删除传播树中拆分出以该子节点为根节点的子树。继而使用该子树的根节点所存储的  $SID$  来计算  $SMAC$ ，并将该  $SMAC$  作为前述子树的根节点的  $SID$ ，从而构造出新的删除传播树。

**STEP5:** 最后，面向其余各个子节点，当前企业节点根据 STEP3 中得到的子节点 ID 向其下一级节点发送已签名的新删除传播树。

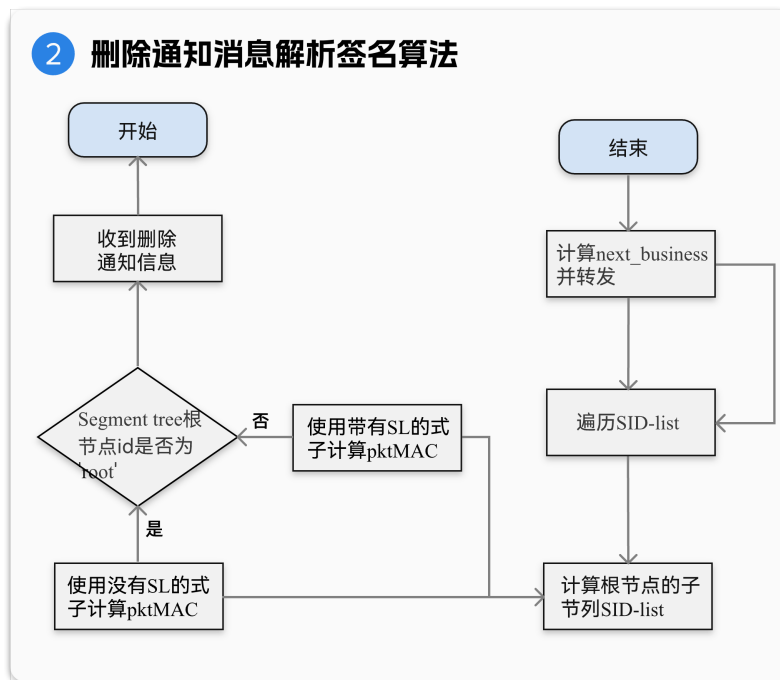


图 3.10 删除通知消息解析签名算法运行图

### 3.3.3.3 删除反馈消息生成算法

企业节点在接收到来自上一级企业节点的删除通知消息后，对删除通知消息做出进一步解析与签名，而后回传删除反馈消息以表明其自身及其子节点均已删除用户所请求删除的目标隐私信息。

**删除反馈消息生成算法步骤具体如下：**

**STEP1:** 当前企业节点从接收到的删除通知消息中提取出删除传播树。利用该删除传播树 `dataTransferPath` 字段中所存储的 `SID` 计算  $ACK = HMAC(session\_key, SID)$ ，同时向删除传播树路径上所有子节点回传该 `ACK` 消息。

**STEP2:** 子企业节点向父企业节点回传删除反馈消息，该删除反馈消息包括当前企业自身 `ACK` 以及其所有子节点 `ACK`。



## 第四章 工程实现

本章简要概述了系统整体的开发流程和前后端技术选择，并逐一介绍了前端以及后端各自的实现方法。最后提供了数据库设计方案和消息类定义方案。

### 4.1 开发业务流程

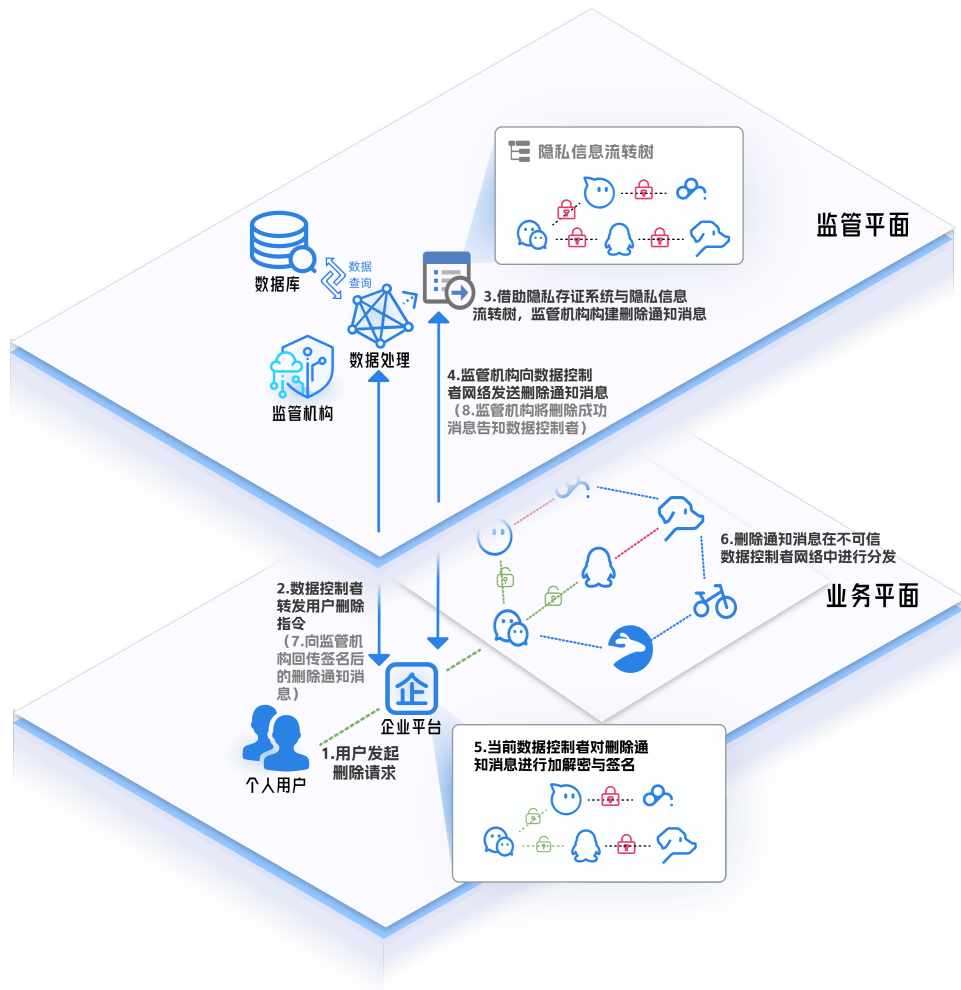


图 4.1 开发业务流程

参考图 4.1, 基于跨域删除通知可信分发协议, 一次删除通知可信分发业务流程可概括如下: 首先, 用户向使用该用户隐私信息的某一数据控制者 (也称源数据控制者) 发起隐私信息删除请求。源数据控制者接收该请求后, 将其转送给中心监管机构。随后中心监管机构接收到来自数据控制者的转送的删除请求, 将使用删除请求解析系统分析用户删除请求, 解析用户删除意图, 并进行删除审核。如满足相关删除条件, 中心监管机构将使用隐私数据流转存证系统生成对应个人隐私

信息流转树，构建删除通知。

随后，系统通过删除指令通知与确认系统，在数据控制者网络中分发删除通知，删除通知经过前向遍历式验证和反向递归式确认后，被相关数据控制者（曾使用过该用户的隐私信息）签名并回送至中心监管机构。中心监管机构验证删除通知上附带的签名，以确保所有使用该用户隐私信息的数据处理域均已完成对应删除操作，最后中心监管机构将删除成功的消息回送给源数据控制者。源数据控制者再将消息告知用户，从而完成一次删除通知过程。

通过以上流程，用户可以发起隐私信息删除请求，数据控制者将请求转送给中心监管机构，并经过中心监管机构解析、审核和验证的步骤，最终完成删除通知的流转分发和确认，确保隐私信息的删除操作得到有效执行。

4.2 开发技术选型

为构建删除通知可信分发系统，实现隐私信息处理可视化平台。在前端侧重于实现用户端，企业端与中心监管机构端这三种交互界面，以模拟并呈现从用户发起隐私信息删除请求，数据处理者接收删除请求并回传至中心监管机构，监管机构审核批准，到数据控制者删除并通知用户的全过程。同时，在后端则侧重于实现前述两种子系统（参考章节 3.1.1）：即删除指令通知与确认系统以及隐私数据流转状态管理与存证系统，以保障删除过程的可信性与完备性。在前后端开发的进程中，考虑使用以下技术方案：

表 4.1 前端技术选型

| 前端技术栈      | 说明                 |
|------------|--------------------|
| Vue3       | 渐进式 JavaScript 框架  |
| Element-UI | 基于 Vue.js 的前端组件库   |
| Ant Design | 前端组件库              |
| Echarts    | 数据可视化图表库           |
| D3.js      | 复杂数据可视化图形库         |
| Ajax       | XML 异步通信交互技术       |
| Saas       | CSS 预处理器           |
| pnpm       | node_modules 包管理工具 |
| Vite       | Web 快速构建工具         |
| ESLint     | 静态代码分析工具           |
| Prettier   | 代码格式化工具            |

表 4.1介绍了开发前端系统所使用的技术方案。Vue 是一款渐进式 JavaScript 框架，具有简单易用，高效灵活的特点，能够与 Vuex 以及 Vue-router 协作，共同构建一个完整的 Web 应用程序。得益于其强大的生态环境，可有效降低系统整体开发成本。Element-UI 和 Ant Design 是基于 Vue.js 的前端基础组件库，而 Echarts 与 D3.js 则是较为流行的数据可视化库，提供了各种图表、图形和可视化组件，使得用户能够直观地了解整个删除过程。Ajax 是一种基于 XML 的异步通信交互技术，用于实现前端与后端的数据交换与实时更新。而 Saas 是一种 CSS 预处理器，能够提供更强大的 CSS 编写方式。pnpm 是一种 node\_modules 包管理工具，能够有效管理项目依赖，减少项目体积与构建时间。而 Vite 是一种 Web 页面快构建工具，提供了快速的开发启动和热模块替换等功能。最后，在项目中引入 ESLint 和 Prettier，有助于统一团队代码风格，进而有效提升团队协作中代码的质量以及可维护性。

表 4.2 后端技术选型

| 后端技术栈        | 说明               |
|--------------|------------------|
| Flask        | 轻量级 PythonWeb 框架 |
| Gevent       | 基于协程的 Python 网络库 |
| Multiprocess | 多进程编程 Python 模块  |
| WSGI         | 接口规范的 Python 标准  |

表 4.2介绍了开发后端系统所使用的技术方案。Flask 是一款基于 Python 的轻量级后端开发框架，能够提供简洁的 API 与灵活的扩展机制，支持 HTTP 请求与响应。Gevent 是一个基于协程的 Python 网络库，允许使用协程来处理并发请求，同时能够使用 greenlet 库与 libev 事件循环实现高性能的异步 I/O 操作。Multiprocess 是 Python 内置的多进程并发模块，用于在多个 CPU 核心上处理并行请求，以充分利用硬件资源，提高系统的吞吐量与并发能力。WSGI(Web Server Gateway Interface)定义了 Python Web 应用程序和 Web 服务器之间的标准接口，该规范使得不同的 Web 框架和服务器能够互相兼容，有助于提升开发过程的灵活性。

4.3 前端实现

4.3.1 前端开发架构

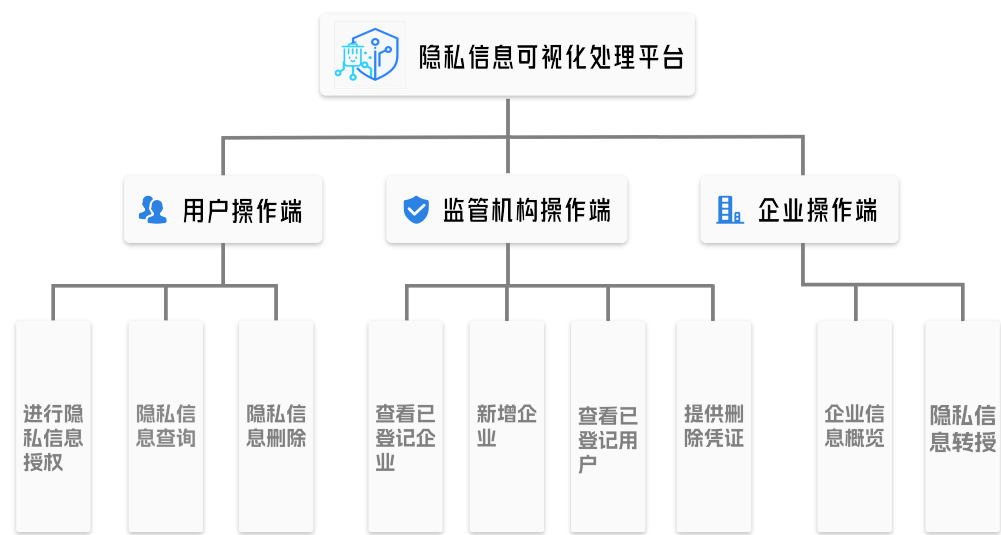


图 4.2 前端功能结构图

前端的开发架构围绕用户端，企业端与中心监管机构端这三种交互界面而展开，各个界面集成了不同的功能与交互方式：

- 1. 用户端：用户端侧重于模拟用户进行隐私信息授权与删除的全过程。面向现实生活中用户的真实删除场景，该部分设置了隐私授权，隐私删除与隐私查询三大业务模块，同时支持用户登录，用户查询删除凭证等功能。
- 2. 企业端：企业操作端则重在模拟数据控制者处理用户删除请求，与中心监管机构通信交互，返回删除结果的全过程。该部分模拟了数据控制者对用户删除请求的处理流程，并通过与中心监管机构的通信来确保删除操作的可信性。
- 3. 中心监管机构端：中心监管机构端则是面向整个删除通知可信分发系统的后台操作界面，用于展示并统计总已登记用户数，总登记企业数，总管理隐私数以及节点服务器配置等关键信息，支持增添数据控制者节点以及用户管理，权限管理等细粒度控制功能，以实现对系统的精细管理。同时支持日志查看功能，允许监管机构查看部分关键操作的日志。

## 4.3.2 代码实现架构

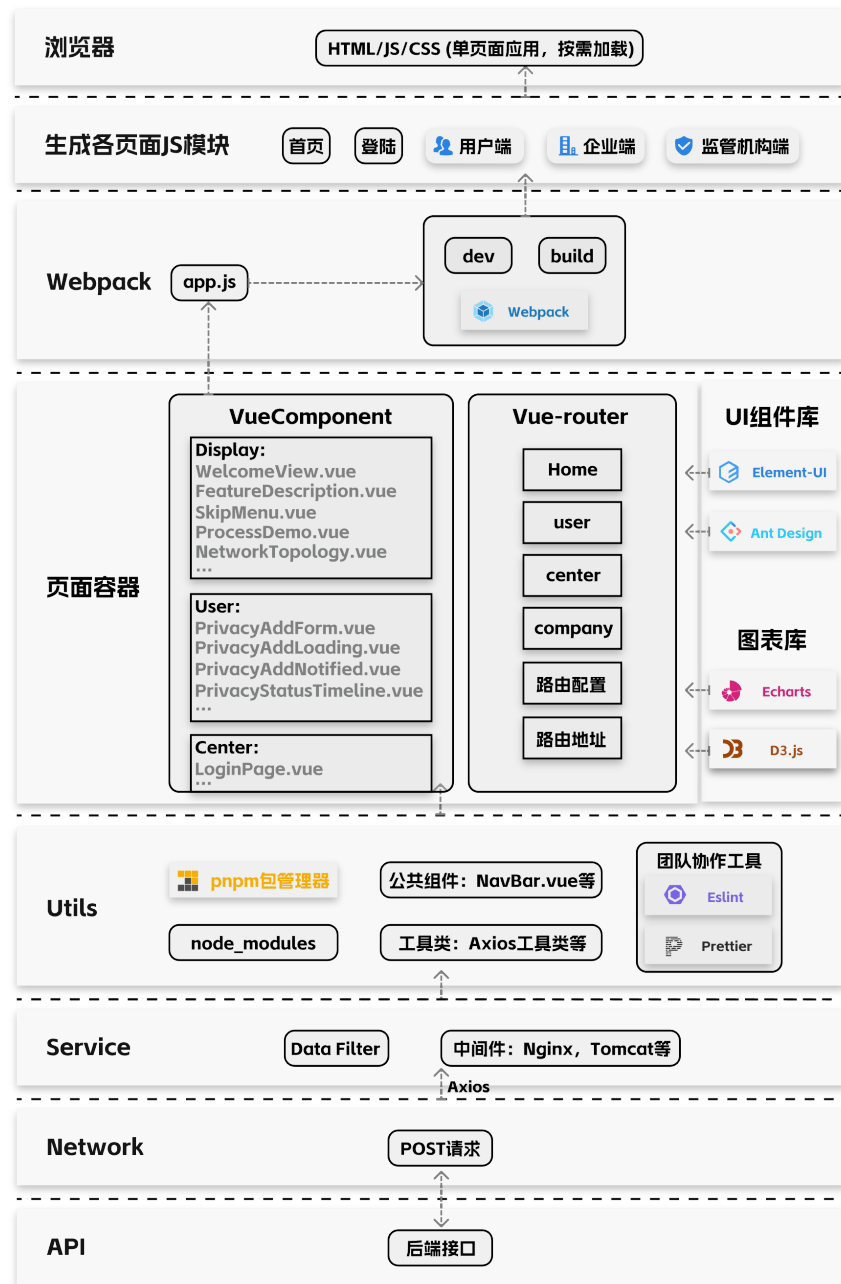


图 4.3 前端代码架构图

在前端代码实现部分，本系统使用 POST 请求与后端进行数据传输，并在 API 层定义了传输格式和交互方式。在 Service 层，通过 Nginx 网关中间件和 Axios 库与上层进行通信，确保数据的可靠传输。在 Utils 层，系统使用 pnpm 包管理器来管理 node\_modules(第三方库)，有助于简化依赖管理和版本控制。同时定义了公共组件（例如 NavBar.vue）与工具类，以提供可重用的代码和功能。同时在 Utils 层，系统还部署了一经典些团队开发工具，使用 Git 以提供版本控制，配置了 Vite 以提

供快速的冷启动与按需编译功能。使用了代码检查工具 ESlint 与代码格式化工具 Prettier，用以提高团队的开发效率、协作能力和代码质量。

在页面容器层，本毕设开发了 User、Company、Center 以及 Display 系列的 Vue 组件，这些组件用于展示不同角色的界面与功能。通过 vue-router 进行路由配置，实现了页面间的导航和跳转。为实现良好的可视化效果，系统使用了 Element-UI、AntDesign、Echarts 和 D3.js 等库。最后，通过 Webpack 打包前端资源，生成各个页面的 JS 模块，并使用浏览器得以呈现。Webpack 提供了模块化管理、代码压缩和性能优化等功能，支持代码分割和懒加载，确保了前端应用的快速载入与高效运行。

4.4 后端实现

本章节将逐步介绍各个子系统的代码实现情况。后端系统由删除指令通知与确认系统以及隐私数据流转状态管理与存证系统构成。

表 4.3 后端代码架构

| 模块名称       | 实现功能   |
|------------|--------|
| Alg        | 算法模块   |
| Data       | 原始数据模块 |
| Database   | 数据库模块  |
| For_test   | 测试模块   |
| Logs       | 日志记录模块 |
| Model      | 模型模块   |
| Simulation | 仿真实验模块 |

表 4.3展示了后端项目的主要代码架构。其中算法模块包含了支持删除通知可信分发协议的多种机制性算法。原始数据模块主要用于记录企业初始密钥，以便计算会话密钥并初始化相关存储数据结构。测试模块和日志记录模块分别用于模块的调试处理和记录代码运行的进度及各种指标，同时为前端交互提供接口。模型模块主要提供了与 ZODB 数据库的交互接口。仿真实验模块基于蓝图模式的 Flask 框架实现，并使用 Flask、Gevent、Multiprocess 和 WSGI 技术组合实现了多线程与高并发功能。

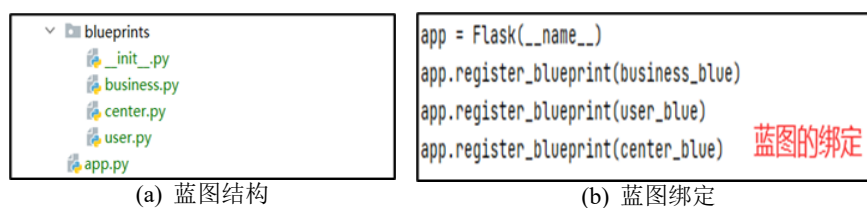


图 4.4 蓝图框架的代码实现

图 4.4展示了蓝图框架的代码实现。在 Flask 项目中，蓝图旨在将应用程序划分为轻量级且可重用的小型模块，它能够提供清晰的分层结构和解耦的设计，使得项目的开发和维护更加高效。

```
def run(MULTI_PROCESS, host, port):  
    if MULTI_PROCESS == False:  
        WSGIServer((host, port), app).serve_forever()  
    else:  
        mulserver = WSGIServer((host, port), app)  
        mulserver.start()  
  
        def server_forever():  
            mulserver.start_accepting()  
            mulserver._stop_event.wait()  
  
        for i in range(cpu_count()):  
            p = Process(target=server_forever)  
            p.start()
```

图 4.5 多线程高并发代码实现

图 4.5展示了多线程高并发的代码实现图。为实现系统整体的多线程与高并发，在一方面，需要将 Flask 应用程序与 Gevent 结合使用，利用 Gevent 提供的协程功能来处理异步的 I/O 操作，例如数据库查询、网络请求等。此外，Flask 还需要与符合 WSGI 标准的服务器进行交互，因此需要一个 WSGI 服务器来处理 HTTP 请求和响应。另一方面，考虑使用 Multiprocess 模块创建多个子进程。每个子进程都可以独立地运行一个 WSGI 服务器实例，并监听不同的端口。这样可以实现多个进程同时处理请求提高并发能力。最后，为了更好地分配请求和提高系统的可靠性，考虑使用负载均衡和反向代理，负载均衡器负责将请求分发给不同的进程和服务，而反向代理服务器则用于缓存静态资源和提供额外的安全性。

```
@user_blue.route('/delete', methods=['POST'])
def delete():
    user_id = request.form["user_id"]
    info_id = request.form["info_id"]
    row_business_id = request.form["row_business_id"]
    data = {
        'user_id': user_id,
        'info_id': info_id,
        'row_business_id': row_business_id
    }
    global_logger.info(f"{user_id}用户发出删除请求")
    r = requests.post('http://127.0.0.1:7777/business/recv_from_user', data=data)
    return "删除完成"
```

图 4.6 蓝图模式下用户发出的删除请求视图函数

图 4.6展示了蓝图模式下用户发出的删除请求视图函数。当用户发送一个删除隐私的 POST 请求时，系统将会将该删除请求转发给相应的数据控制者。在 Flask 框架下，用户蓝图将会调用数据控制者蓝图，而数据控制者蓝图则会进一步访问监管机构蓝图，进而启动整个删除通知可信分发系统。

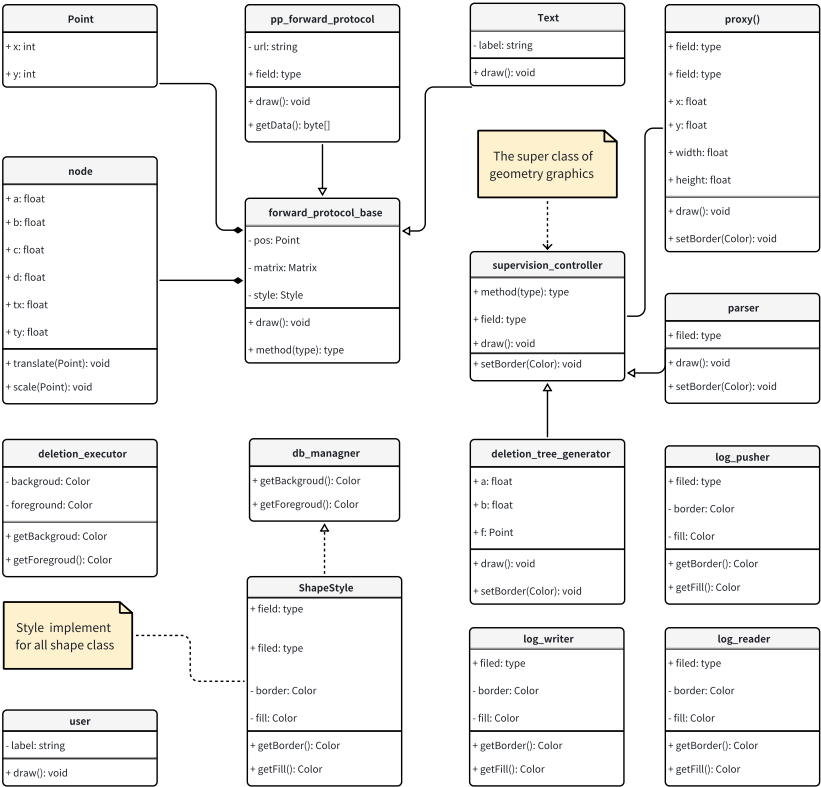


图 4.7 UML 类图

图 4.7为后端开发 UML 类图，描述了后端系统中各种类、接口、协作以及其各自间的关系。

#### 4.4.1 删除指令通知与确认系统

该子系统基于删除通知可信分发协议进行开发，负责承担业务与监管平面间交互通信，解析用户删除请求，删除通知的生成与下发以及删除通知验证等多项关键任务。能够忠实地执行删除通知可信分发的全套业务流程，确保了数据控制者能够与监管机构进行安全的数据交互，通过协议进行删除操作的确认和验证，进而保障数据的隐私与安全。

在代码部分，将以开发业务流程 (参考章节 4.1) 为例，逐步介绍删除通知可信分发的全过程。

```
for i, child_id in enumerate(children_id):
    if position[0] == 1 and position[1] == 1:
        pktMAC = bytes.hex(hmac(key=bytes.fromhex(key), msg=payload.encode()))
        IP_last = bytes.hex(padPKCS7(b'center'))
        if child_id[0] != 'L':
            IP_next = bytes.hex(padPKCS7(child_id.encode()))
        else:
            IP_next = bytes.hex(padPKCS7(b'center'))
        SID = hex_xor(IP_next, bytes.hex(hmac(bytes.fromhex(key), (IP_last + pktMAC + time_stamp + str(i+1)).encode()))))
    else:
        msg = payload + get_position_node(SL, position)
        pktMAC = bytes.hex(hmac(key=bytes.fromhex(key), msg=msg.encode()))
        IP_last = bytes.hex(padPKCS7(parent_id.encode()))
        if child_id[0] != 'L':
            IP_next = bytes.hex(padPKCS7(child_id.encode()))
        else:
            IP_next = bytes.hex(padPKCS7(b'center'))
        SID = hex_xor(IP_next, bytes.hex(
            hmac(bytes.fromhex(key), (IP_last + pktMAC + time_stamp + str(i + 1)).encode()))))
    segment_tree.create_node(identifier=bytes.fromhex(SID), parent=get_position_node(segment_tree, position))
    SL.create_node(identifier=bytes.hex(hmac(bytes.fromhex(key), bytes.fromhex(SID))),
                    parent=get_position_node(SL, position))
```

图 4.8 删除通知构建

图 4.8展示了构建删除通知的函数实现。当监管机构接收到来自于数据控制者转发的用户删除请求时，会构建对应的删除通知消息，以通知其余相关数据控制者 (这些数据控制者同样使用了该用户隐私信息) 进行删除。该消息中包含了用于删除与解析的 `dataTransferPath` 字段以及后续验证删除凭证的 `delSignatureDict` 字段。

字段生成逻辑如下: 监管机构依据从数据控制者处接收的相关参数 (参考 4.4.2), 从隐私数据流转状态管理与存证系统中获取隐私信息流转树，再依据隐私信息流转树计算出 `pkt_mac`，使用 PKCS7 方法填充 `ip_last`，进而结合时间戳 `time_stamp` 与元素在树中的层次位置 `i` 计算出 `SID`，从而构建出 `dataTransferPath` 字段与 `delSignatureDict` 字段。

```
secret_i = busid_to_secret(to_bus_id)
key_i = hashlib.sha256((secret_i + time_stamp).encode()).hexdigest()  密钥获取
```

图 4.9 获取解析密钥

```

if get_position_node(segment_tree, (1, 1)) == "root":
    pkt_mac_i = bytes.hex(hmac(bytes.fromhex(key_i), payload.encode()))
else:
    pkt_mac_i = bytes.hex(hmac(bytes.fromhex(key_i), (payload + get_position_node(segment_tree, (1, 1))).encode()))
sid_list = segment_tree.children(get_position_node(segment_tree, (1, 1)))
next_bus_id_list = []
flag = 1
for i in sid_list:
    last_bus_ip = bytes.hex(padPKCS7(from_bus_id.encode()))
    bus_ip = hex_xor(i.identifier, bytes.hex(
        hmac(bytes.fromhex(key_i), (last_bus_ip + pkt_mac_i + time_stamp + str(flag)).encode())))
    bus_id = unpadPKCS7(bytes.fromhex(bus_ip)).decode()
    next_bus_id_list.append(bus_id)
    flag = flag + 1

```

删除树的解析

图 4.10 解析删除通知

图 4.9与图 4.10分别展示了获取解析密钥与解析删除通知的函数实现。在前向遍历式验证机制作用下，数据控制者的主要任务是解析删除通知消息中的 data-TransferPath 字段，同时进行删除操作并通知后续数据控制者节点完成删除任务。首先，数据控制者获取企业自身密钥，再根据时间戳 time\_stamp 和企业密钥的哈希值解析出会话密钥。

紧接着需要进行删除通知的解析，删除通知解析逻辑如下：首先需要判断头结点是否为根节点（root），根据不同情况计算出 pkt\_mac 值，然后使用计算得到的 pkt\_mac 计算 bus\_ip。此时还需对 bus\_ip 进行 PKCS7 填充的逆运算以正确计算出 bus\_id。得到 bus\_id 后，将其添加至 next\_bus\_id\_list 中，以备后续的转发操作使用。

```

for i in next_bus_id_list:
    if i == "center":
        break
    elif int(i[1:]) < 1000 or int(i[1:]) > 1099:
        return tuple(['N01'])*4 # 下一条解密出错
    elif i == from_bus_id:
        return tuple(['N02'])*4 # 路径重合出错

```

图 4.11 路径解析验证

```

next_segment_tree_list = []
for i in sid_list:
    tree = treelib.Tree()
    new_mac = bytes.hex(hmac(bytes.fromhex(key_i), bytes.fromhex(i.identifier)))
    tree.create_node(identifier=new_mac)
    subtree = segment_tree.subtree(i.identifier)
    for j in subtree.children(subtree.get_node(i.identifier).identifier):
        origin_tree = segment_tree.subtree(j.identifier)
        tree.paste(new_mac, origin_tree)
    next_segment_tree_list.append(tree)

```

重组转发

图 4.12 反向递归式回传

图 4.11与图 4.12分别展示了路径解析验证与反向递归式回传的函数实现。当

一数据控制者接收到另一数据控制者的删除通知消息时，需要验证 `next_bus_id_list` 中 `bus_id` 的正确性。在代码实现中已经考虑了两种情况：下一跳解密异常与路径重合异常，以上措施能够有效应对前述威胁模型中的恶意攻击行为。

在反向递归式确认机制下，数据控制者需要使用该跳转通知数据控制者的 `SID` 与其会话密钥进行签名操作，同时将签名后的结果与接收到的隐私流转树中对应数据控制者的子树连接起来，并将其转发给下一跳数据控制者进行解密操作。

#### 4.4.2 隐私数据流转状态管理与存证系统

为进一步支持中心监管机构进行检索，以生成对应删除通知，本毕设设计并实现了隐私数据流转状态管理与存证系统。该系统模块负责存储用户隐私信息的流转情况，并为删除指令通知与确认系统构建删除通知提供相应决策依据。该存证系统能够高效响应生成并下发对应的删除通知树，便捷地验证可信删除凭证的正确性，为删除指令的通知与确认工作提供安全保障。

在代码实现部分，为应对快速增长的数据规模，同时有效降低监管机构的存储负担，本方案考虑使用布谷鸟过滤器作为存储数据结构，这是一种基于散列函数的高效概率型数据结构。方案预设每个合法注册的数据控制者都于中心监管机构处开辟一块存储空间，中心监管机构再使用 `ZODB` 以存储相应数据控制者所属的布谷鸟过滤器。`ZODB` (`Zope Object Database`) 的设计理念是将 `Python` 对象直接存储到数据库中，而无需像关系数据库那样将对象映射为表和行。这一设计理念使得开发者可以使用普通的 `Python` 对象和数据结构，而不必关心数据的序列化和反序列化过程。具体实现如下：

```
storage = ZODB.FileStorage.FileStorage('../database/business_cuckoofilter.fs')
compressed_storage = zc.zlibstorage.ZlibStorage(storage) # 使用zlibstorage来压缩数据记录
db = ZODB.DB(compressed_storage)
connection = db.open()
root = connection.root()
```

图 4.13 初始化数据库

```
for i in range(100):
    business_id = 'b' + str(1000 + i)
    c = CuckooFilter(capacity=200, fingerprint_size=10)
    root[business_id] = c
transaction.commit()
```

图 4.14 初始化布谷鸟过滤器

```

df = pd.read_excel('../data/infoauthset_datageneration.xlsx')
interface = MyZODB('../database/business_cuckoofilter.fs')
for index, row in df.iterrows():
    for k, v in eval(row[2]).items():
        info_id = info_id_map[k]
        interface.add_element(row[4], row[1], info_id, row[5], row[3])
interface.close()

```

图 4.15 初始化转授记录及数据库接口实例

图 4.13, 图 4.14 以及图 4.15 展示了隐私数据流转状态管理与存证系统的初始化过程, 首先初始化数据库并使用 zlibstorage 压缩数据记录, 随后初始化数据控制者所属布谷鸟过滤器, 完成对最初布谷鸟过滤器的创建工作, 并将其存储于数据库中。最后初始化转授数据并创建操作数据库接口实例, 以供后续使用。

```

def contains(self, user_id, info_id, row_business_id):
    fp, i1, i2 = self._get_fp_and_index_pair(user_id, info_id, row_business_id)
    b1, b2 = self.__buckets[i1], self.__buckets[i2]
    return (b1.get_fingerprint_index(fp) > -1) or (b2.get_fingerprint_index(fp) > -1)

```

图 4.16 查询流转信息

```

def insert(self, user_id, info_id, row_business_id, last_business_id):
    fp, i1, i2 = self._get_fp_and_index_pair(user_id, info_id, row_business_id)
    MU = fp + last_business_id.encode('utf-8')
    if self._insert(MU, i1) or self._insert(MU, i2):
        return True
    i = random.choice([i1, i2])
    for k in range(MAX_CUCKOO_COUNT):
        MU, j = self.__buckets[i].swap(MU)
        i = self._get_alter_index(MU, i)
        if self._insert(MU, i):
            return True
    return False

```

获取元素指纹和位置

插入元素

图 4.17 插入流转信息

```

def delete(self, user_id, info_id, row_business_id):
    fp, i1, i2 = self._get_fp_and_index_pair(user_id, info_id, row_business_id)
    b1, l1 = self._delete(fp, i1)
    if b1 == True:
        return b1, l1
    b2, l2 = self._delete(fp, i2)
    if b2 == True:
        return b2, l2
    else:
        return False, None

```

元素的查找

元素删除

```

def _delete(self, fp, i):
    # 删除 i 位置中的字节串
    b, l = self.__buckets[i].delete(fp)
    if b:
        # with self.__lock:
        self.__count -= 1
        return True, l
    return False, l

```

(a) 隐私信息删除核心函数

(b) \_delete 方法

图 4.18 删除流转信息

图 4.16, 图 4.17 以及图 4.18 展示了布谷鸟过滤器查询, 插入以及删除用户隐私信息的全流程, 查询操作以信息三元组 (user\_id, info\_id, source\_business\_id) 为输入, 查询逻辑如下: 将待查询的元素通过散列函数映射到对应的桶和位置, 并检查该位置上是否存在相同的指纹。如果存在, 则元素可能在过滤器中; 如果不存在, 则

元素一定不在过滤器中。而插入操作则在信息三元组的基础上，新增了目标数据控制者 ID 作为输入。过滤器使用信息三元组计算出元素的指纹与位置，进而利用目标数据控制者 ID 与指纹进行后续查找。删除操作则是在查询操作的基础上得以实现。同样以信息三元组为输入，由于布谷鸟过滤器是近似集合过滤器，没有直接的删除操作。所以在函数中定义了 `_delete` 方法，并将对应桶的 `_count` 属性-1。

## 4.5 数据库设计

删除通知可信分发系统整体所属的后端数据库名为 **TESS(Trusted Evidence Storage System)**，分别以 `user_ID`，`business_ID` 以及 `regulatory_ID` 为主键，数据类型皆为 `varchar(255)`。接下来介绍各张表的结构以及功能。

表 4.4 数据控制者本地信息存证数据表

| 列名                              | 数据类型                      | 说明       |
|---------------------------------|---------------------------|----------|
| <code>user_ID</code>            | <code>varchar(255)</code> | 用户 ID    |
| <code>info_ID</code>            | <code>varchar(255)</code> | 流转信息 ID  |
| <code>delete_method</code>      | <code>varchar(255)</code> | 删除方法     |
| <code>delete_granularity</code> | <code>varchar(255)</code> | 删除粒度     |
| <code>delete_notifytree</code>  | JSON                      | 删除确认树    |
| <code>del_signature_dict</code> | JSON                      | 删除确认签名字典 |
| <code>trigger_type</code>       | <code>varchar(255)</code> | 触发器类型    |
| <code>source_domain</code>      | <code>varchar(255)</code> | 源数据控制域   |
| <code>time_limit</code>         | <code>varchar(255)</code> | 时间限制     |
| <code>count_limit</code>        | <code>int</code>          | 计数限制     |

**表 4.4 为本地信息存证的数据表。**当源数据控制者节点转发删除请求时，会于本地先行存证已有删除信息，该行为会新建数据表，以 `user_ID` 为主键。当删除通知全部完成时，数据控制者能够接收到所有的删除通知确认，同时根据 `user_ID` 查询信息表，依据 `info_ID` 查询对应信息条目（即使用 `user_ID` 与 `info_ID` 可唯一地确定或定位某条隐私信息），随后更新本地信息存证表中的 `del_signature_dict` 字段（签名信息）。

在数据控制者向中心监管机构进行推送时，首先查询本地信息存证的数据库。依据相关本地存证信息，数据控制者组织好对应内容，按照既定的通信协议，再向中心监管机构进行推送。

表 4.5 中心监管机构存储信息表

| 列名                 | 数据类型         | 说明        |
|--------------------|--------------|-----------|
| affairs_id         | varchar(255) | 唯一事务标识符   |
| user_id            | varchar(255) | 用户 ID     |
| info_id            | varchar(255) | 流转信息 ID   |
| count_limit        | int          | 计数限制      |
| source_bus_id      | varchar(255) | 源数据控制者 ID |
| delete_method      | varchar(255) | 删除方法      |
| delete_granularity | varchar(255) | 删除粒度      |

表 4.5为中心监管机构存储信息表，当中心监管机构需要插入流转记录时，首先查询以 info\_id 为标识的数据表，若未查询到对应数据表，则创建一个以 info\_id 为标识的数据表，同时插入 from\_bus\_id 和 to\_bus\_id。若成功查询到相应的数据表，则在表中直接插入 from\_bus\_id 和 to\_bus\_id。

此外，重点介绍 count\_limit 字段的意义：在首次数据流转时，将新建一个中心监管机构存储信息表，同时将隐私信息存储于该存储信息表中。此后每当在存储信息表中插入一条流转信息时，中心监管机构会查询该表中所记录的数据流转次数，并同时查询该表中用户在首次数据流转时设定的流转次数限制”count\_limit”字段。如果流转次数限制大于等于数据流转次数，则允许数据继续流转；反之，则不允许数据流转。经过以上步骤，一次计次检查流程已顺利完成。

表 4.6 用户注册数据表

| 列名           | 数据类型         | 说明       |
|--------------|--------------|----------|
| user_ID      | varchar(255) | 表主键      |
| create_time  | varchar(255) | 用户创建时间   |
| update_time  | timestamp    | 用户信息更新时间 |
| mail_address | varchar(255) | 邮箱地址     |

表 4.6为用户注册信息数据表，包括用户 ID，用户创建时间，用户信息更新时间以及用户邮箱地址（可选）。

表 4.7 数据控制者节点注册表

| 列名          | 数据类型         | 说明     |
|-------------|--------------|--------|
| business_ID | varchar(255) | 表主键    |
| create_time | timestamp    | 创建时间   |
| update_time | timestamp    | 信息更新时间 |

表 4.7为数据控制者注册信息数据表，包括数据控制者 ID，数据控制者创建时间以及数据控制者信息更新时间。

表 4.8 数据控制者节点在线状态表

| 列名               | 数据类型          | 说明    |
|------------------|---------------|-------|
| business_ID      | varchar(255)  | 表主键   |
| online_status    | tinyint       | 在线状态  |
| count_of_users   | int           | 下辖用户数 |
| count_of_privacy | int           | 下辖隐私数 |
| server_name      | varchar(255)  | 服务器名称 |
| operatin_system  | varchar(255)  | 操作系统  |
| IP_public        | varcharR(255) | 公网 IP |
| IP_private       | varchar(255)  | 内网 IP |
| server_config    | varchar(255)  | 服务器配置 |

表 4.8为数据控制者节点在线状态表，用于记录各个数据控制者服务器的连接服务状况（默认情况下，预设单一数据控制者对应单一服务器）。同时可用于记录对应服务器用户管理，隐私管理，软硬件配置以及网络配置等情况。在线状态分为在线与离线，使用 tinyint 数据类型进行存储，后端对应的实体类属性为 Boolean。

表 4.9 中心监管机构数据表

| 列名                   | 数据类型         | 说明      |
|----------------------|--------------|---------|
| regulatory_ID        | varchar(255) | 表主键     |
| online_status        | tinyint      | 在线状态    |
| sum_of_totalusers    | int          | 总已登记用户数 |
| sum_of_totalprivacy  | int          | 总管理隐私数  |
| sum_of_totalbusiness | int          | 总登记企业数  |

表 4.9为中心监管机构数据表，用于在中心监管机构层面对整个系统进行归纳

与统计，可用于记录总已登记用户数，总管理隐私数以及总登记企业数等关键统计信息。

4.6 消息类

本系统各子系统之间通信遵循删除通知可信分发协议，统一使用固定格式（参考表 4.10、表 4.11、表 4.12以及表 4.13）。本章节主要介绍**删除请求消息**，**隐私流转树**，**删除通知确认消息**以及**删除通知签名**其各自结构。消息时间与服务器同步，均为 UTC+8 时区时间。所有消息类均为 **JSON 格式**，使用 Post 请求在数据控制者网络之间传输。

表 4.10 删除请求消息结构

| 列名                 | 数据类型         | 说明      |
|--------------------|--------------|---------|
| affairs_id         | varchar(255) | 唯一事务标识符 |
| user_id            | varchar(255) | 用户 ID   |
| info_id            | varchar(255) | 流转信息 ID |
| time_limit         | varchar(255) | 时间限制    |
| count_limit        | int          | 计数限制    |
| delete_method      | varchar(255) | 删除方法    |
| delete_granularity | varchar(255) | 删除粒度    |

表 4.10展示了**删除请求消息结构**。删除请求消息由数据控制者发起，进而被呈送给中心监管机构。用户发起的删除请求在数据控制者处得到解析，因此设置了 delete\_method 与 delete\_granularity 字段以提供细粒度的删除选项。为抵御不可信网络中存在的恶意攻击行为，进而设置了 count\_limit 与 time\_limit 字段加以限制。

表 4.11 dataTransferPath 隐私流转树结构

| 列名          | 数据类型         | 说明      |
|-------------|--------------|---------|
| from_bus_ID | varchar(255) | 源数据控制者  |
| to_bus_ID   | varchar(255) | 目标数据控制者 |

表 4.11展示了**隐私信息流转树结构**。隐私流转树是删除通知可信分发协议的核心概念之一(参考章节 3.2)，它将删除通知转发问题形式化为个人信息流转树遍历验证问题(参考章节 2.1)。在具体实现过程中，隐私流转树被用于辅助数据控制

者和中心监管机构进行数据存储 (参考表 4.5) 以及构建删除通知确认消息与删除通知签名。(参考表 4.12及表 4.13)

表 4.12 删除通知确认消息结构

| 列名               | 数据类型         | 说明       |
|------------------|--------------|----------|
| affairs_id       | varchar(255) | 事务标识符    |
| from_bus_id      | varchar(255) | 源数据控制者   |
| to_bus_id        | varchar(255) | 目标数据控制者  |
| user_id          | varchar(255) | 用户 ID    |
| info_id          | varchar(255) | 流转信息 ID  |
| dataTransferPath | JSON         | 隐私流转树    |
| delSignatureDict | JSON         | 删除通知签名字典 |

表 4.12展示了删除通知确认消息结构。其中，dataTransferPath 字段 (即隐私流转树) 决定了删除通知确认消息在数据控制者网络中的传输路径。删除通知确认消息借助前向遍历式验证机制 (参考章节 3.2.1) 被分发至相关数据控制者节点。紧接着，它通过反向递归式确认机制收集数据控制者网络中各个相关节点的签名，最后被回送至中心监管机构，以待进一步处理。

表 4.13 delSignatureDict 删除通知签名结构

| 列名               | 数据类型         | 说明      |
|------------------|--------------|---------|
| affairs_id       | varchar(255) | 事务标识符   |
| from_bus_id      | varchar(255) | 源数据控制者  |
| to_bus_id        | varchar(255) | 目标数据控制者 |
| user_id          | varchar(255) | 用户 ID   |
| info_id          | varchar(255) | 流转信息 ID |
| dataTransferPath | JSON         | 隐私流转树   |

表 4.13展示了删除通知签名结构。删除通知签名字典是反向递归式确认机制 (参考章节 3.2.2) 的关键性内容，也是删除通知确认消息的重要组成部分，该签名机制旨在保障删除通知的完备性，确保所有涉及数据控制者进行数据留痕，以实现本毕设跨域删除通知可信分发的研究目标。



## 第五章 系统测试

本章为系统测试，首先进行了对删除通知可信分发协议的安全性分析与性能评估，简述了性能测试的实验环境，并对协议的性能测试结果进行了分析总结。其次，对删除通知可信分发系统开展了全方位测试，介绍了系统测试数据的设计与生成方案，以及性能测试的测试环境与结果分析。最后，以隐私信息可视化平台为例，对整个系统进行了功能正确性测试，以验证系统的各项功能是否符合设计要求。

### 5.1 安全性分析

本小节对删除通知可信分发协议进行了安全性分析，协议综合使用了加密、消息认证码以及时间戳等措施，阻止了篡改、假冒、路径重定向、重放以及丢弃攻击等恶意攻击行为的发生，保障了删除通知分发的可信与完备性。

1. **篡改**：恶意攻击者 Eve 拦截了数据控制者转发的删除通知，同时将 Alice 的 ID 篡改为 Bob 的 ID，并将其发送给目标数据控制者。目标数据控制者通过比对认证标签，识别出标签中的不匹配之处，表明接收到的删除通知本身存在异常。
2. **假冒**：恶意攻击者 Eve 获取了 Alice 的身份信息 ID，试图伪装成 Alice 个人信息流转树中某一跳数据控制者，并向持有 Alice 隐私信息的目标数据控制者发送虚假的删除通知请求。然而，Eve 并不知道数据控制者间的会话密钥  $k$ ，使用了一个不是该会话密钥的其他字符串，作为标签生成算法中的密钥进行输入。目标数据控制者通过标签进行验证，检测到不匹配之处，成功识别出上述攻击行为。
3. **路径重定向**：恶意攻击者 Eve 截获了 Alice 发送的删除通知请求，篡改了该请求的目的地址后发送到了非目标数据控制者处，非目标数据控制者通过比对标签 Auth，发现标签不匹配，表明接收到的删除通知本身存在异常。
4. **重放攻击**：恶意攻击者 Eve 拦截并保存了数据控制者之间转发的删除通知，而未进行任何修改。在一段时间后，删除通知被重新发送给目标数据控制者。目标数据控制者通过检测时间戳，成功识别出重放攻击行为。

5. **丢弃攻击**: 数据控制者 M 将 Alice 的删除通知转发给数据控制者 N, 但恶意攻击者 Eve 拦截并丢弃了该通知, 然后向数据控制者 M 发送了虚假的删除成功确认消息。数据控制者 M 对响应消息进行验证, 发现不匹配, 表明删除通知的发送存在异常。

## 5.2 删除通知可信分发协议性能分析

### 5.2.1 实验测试环境

在业务层面上, 本毕设使用了基于 Python 的 Sanic 框架<sup>1</sup>, 同时利用了异步 I/O 技术与 RESTful API, 开发实现了前述删除通知可信分发协议。

实验使用 containernet 进行, 这是一个基于 mininet 的网络仿真工具, 通过 Docker<sup>2</sup>集成进行了增强。在本次毕设的仿真网络环境中, 每个 Docker 容器充当一个数据控制者。为了模拟删除通知的分发过程, 并准确评估方案在真实网络基础设施中的性能, 使用了来自 Internet Topology Zoo<sup>[27]</sup>的几个经典网络拓扑结构, 包括 Chinanet、Karen、Lambdanet 和 Viatel。这些网络拓扑在 Topology Zoo 中以 GraphML 格式描述。每个 GraphML 文件包含了各种网络节点和相互连接。基于 Containernet 的网络仿真通过将 GraphML 解析为 Docker 容器和虚拟网络链接来启动。

仿真程序于托管在阿里云上的虚拟机 (VM) 上执行。该虚拟机的配置如下: 一个 ecs.g7t.16xlarge 云服务器, 支持用于可信执行环境的 Intel SGX, 并配备 128GB 的加密内存。此外, 服务器还配置有 64 个虚拟 CPU 核心 (vCPUs), 256GB 内存和 32GiB 网络带宽。

### 5.2.2 性能评估

为评估真实删除场景下**网络功能模块**(参考章节 3.2.4, 网络功能模块结合了可信执行环境, 用于实施删除通知可信分发协议) 的具体资源开销, 本次毕设设计了一系列实验, 围绕 **CPU 利用率与内存占用**两个关键指标展开。确保通信安全的一种常见做法是使用端到端加密技术 (例如 TLS), 因此考虑将本毕设方案与 TLS 方案分别在 CPU 利用率及内存占用这两个维度上进行对比分析。考虑到使用可信执行环境 (TEE) 能够提升会话密钥的安全性, 本小节将进一步探讨在 TEE 内外实现该方案对资源消耗的影响。有鉴于此, 考虑使用自适应采样机制以降低数据控

<sup>1</sup><https://github.com/sanic-org/sanic>

<sup>2</sup><https://github.com/containernet>

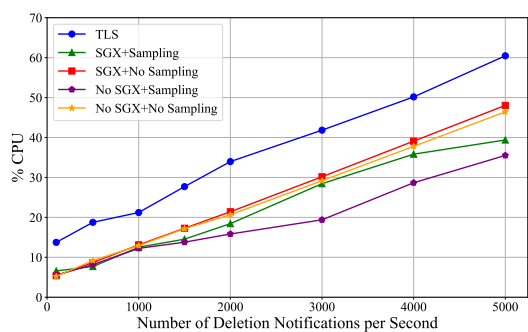
制者的负担，将分别测试使用与禁用该机制时数据控制者的 CPU 利用率与内存占用情况，旨在证明自适应采样机制在减轻资源消耗方面的有效性。最后，为证明网络功能模块的普适性，随机选取四个网络拓扑：Chinanet、Karen、Lambdanet 和 Viatel，观察在各个拓扑上该方案的表现是否相似。

性能评估的结果显示在图 5.1 中。在每个子图中，X 轴代表每个数据控制者每秒接收到的删除通知数量，即删除通知的强度。同时，Y 轴描述了相应的 CPU 利用率与内存占用情况。

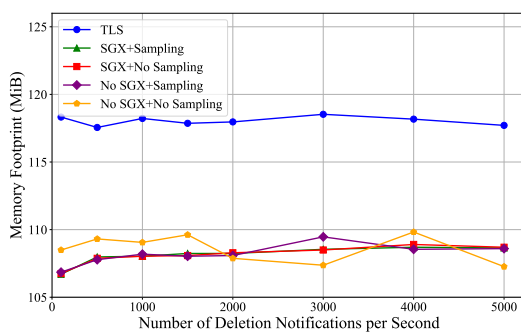
首先，参见图 5.1(a)，随着删除通知强度的不断提升，观察对比 TLS 方案与本毕设其他方案在 CPU 利用率方面的情况。观察到 TLS 方案其 CPU 利用率随着每秒钟删除通知数量的增加而显著升高，CPU 利用率在每秒处理 5000 条删除通知时接近 60%，远高于本毕设其他方案。这主要是因为 TLS 依赖于计算成本较高的非对称加解密过程，而本毕设的方案基于对称加密，能极大减少加解密所需的 CPU 资源，以保证该方案在数据控制者的负载较高时仍然能保持较高的性能。

其次，观察采用 SGX 方案与禁用 SGX 方案在 CPU 利用率方面的差异，方案使用 SGX 将导致 CPU 利用率小幅上升，这主要是因为 SGX 在提供 TEE 的同时，引入了一定的 I/O 交换负担。然而，采用 SGX 的方案在 CPU 利用率小幅上升的同时，却极大地提升了模块的整体安全性，这要归功于该方案将网络功能模块的核心功能（即验证部分）部署在 SGX 内，因此有效减少了可信执行环境内的计算量。

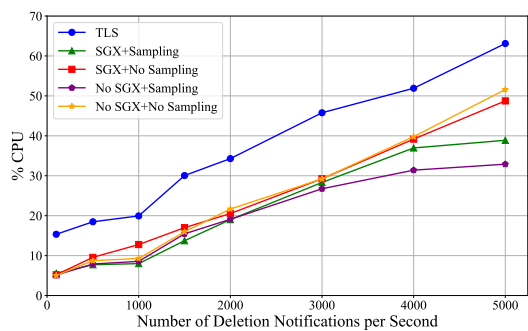
最后，观察采样方案与不采样方案对 CPU 利用率产生的影响，当每秒删除通知数量超过 1000 条时，采样方案与不采样方案对 CPU 利用率的影响开始显现出明显差异。随着删除通知数量的进一步增加，使用自适应概率采样机制的方案其 CPU 利用率明显低于不使用采样的方案。这是因为自适应概率采样机制能够减少处理的数据量，进而有效降低了 CPU 的工作负载。特别是当删除通知数量达到一定阈值后，使用自适应概率采样机制的效益变得尤为明显。因此，数据控制者可以根据自身的具体情况灵活设置采样参数，以此来平衡系统性能和数据处理需求。同时，根据安全性的具体要求，数据控制者还能够选择适当的采样策略或决定是否采样，确保在优化性能的同时不牺牲系统的安全性。



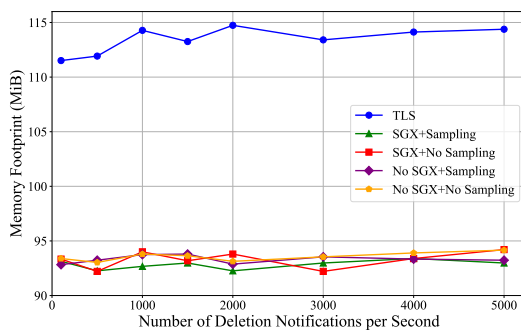
(a) Chinanet 拓扑下的 CPU 利用率



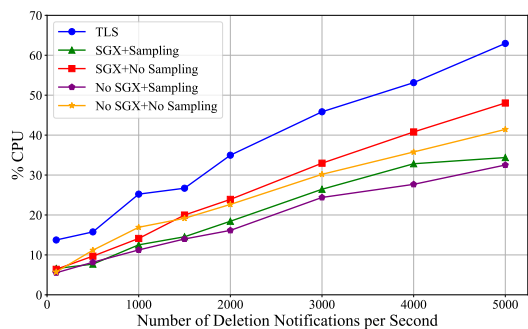
(b) Chinanet 拓扑下的内存占用



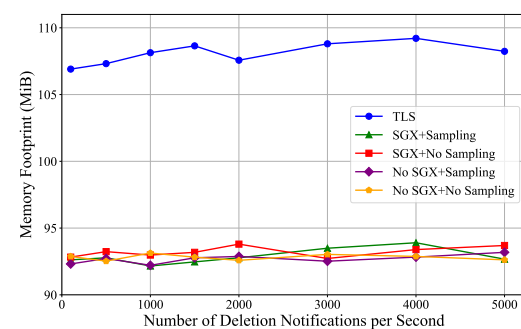
(c) Karen 拓扑下的 CPU 利用率



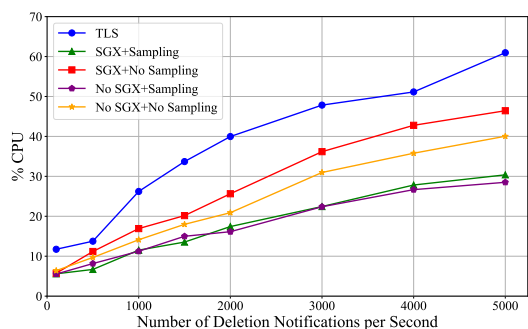
(d) Karen 拓扑下的内存占用



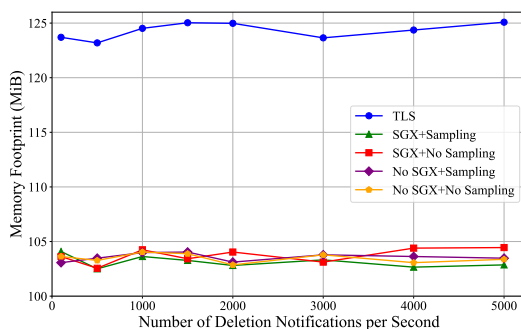
(e) Lambdanet 拓扑下的 CPU 利用率



(f) Lambdanet 拓扑下的内存占用



(g) Viatel 拓扑下的 CPU 利用率



(h) Viatel 拓扑下的内存占用

图 5.1 不同网络拓扑下，删除通知强度相关的 CPU 利用率与内存占用情况

接下来，参见图 5.1(b)，随着删除通知强度的不断提升，观察对比 TLS 方案与本毕设其他方案在内存占用方面的情况。TLS 方案的内存占用均保持在 118MiB 左

右，而本毕设提出的方案其内存占用均保持在 107MiB 左右，显著低于 TLS 方案。这主要是因为 TLS 在通信过程中持续使用非对称加解密算法，需要更多的内存来存储密钥与中间状态信息。而 SGX 的使用与否对内存占用影响不大，因为 SGX 专注于保护代码与数据的安全，不会对模块整体的内存占用情况产生显著影响。同时观察到抽样机制的使用与否对内存占用影响不大，因为抽样策略旨在减少需要处理的删除通知数量，主要影响 CPU 负载与处理时间，而非内存占用。不论何种方案，整体内存占用均维持在一个稳定区间内，这是因为在实验场景下对网络功能模块的操作均为 I/O 密集型操作，其内存占用不会随着删除通知数量的增加而显著增加。同时观察到 CPU 利用率会随着删除通知数量的增加而显著增加，这是因为 I/O 密集型操作经常使用异步或非阻塞 I/O 操作来提高效率，其允许 CPU 在等待 I/O 操作完成时继续处理其他删除通知，从而提高应用的整体性能。随着处理的 I/O 操作数量增加，其 CPU 的利用率也会对应提高。

在其余三个网络拓扑 (参见图 5.1(c)与图 5.1(h)) 下的实验显示出了与 Chinanet 拓扑下实验类似的结果，这表明本毕设提出的方案具有良好的普适性与稳健性。这主要是因为删除通知可信分发协议 (参考章节 3.2) 的设计，使得机制本身与具体的网络拓扑解耦，数据控制者在每个节点上独立地执行，只关注从一个节点到下一个节点的数据验证。有鉴于此，该方案可以适应用任意网络拓扑与个人数据流转树。

### 5.3 删除通知可信分发系统测试

#### 5.3.1 测试数据

##### 5.3.1.1 测试数据设计

为确保测试数据能够涵盖各种可能的使用场景，以检验系统在不同条件下的适应能力与功能正确性。本次毕设设置了授权信息数目，拓扑结构以及信息转授量三个测试用例关键指标。从授权隐私信息数目角度出发，将考虑用户初始授权信息量较少或较多的情况，以测试系统在极限条件下的表现（包括极端及异常情况）。从拓扑结构角度出发，将考虑仅一层结构（包括深度仅一层，但广度极广的极端测试样例）对系统的影响。与此同时，需要测试系统对于具有非常深的转授树结构的处理能力，特别需要考虑到长链情况（广度为一，但是深度非常深）。最后还要测试系统对于复杂树形结构（设置的广度和深度均一般）的处理性能。从信息转授量角度出发，在真实删除场景下发生于数据控制者内部的隐私信息转授

并非均为初始隐私量的全集，可能仅为一部分。因此对于同样的个人信息流转授权树，还需考虑每次转授的信息量是否为初始信息量的全集。测试数据的类别划分方案如图 5.2所示：

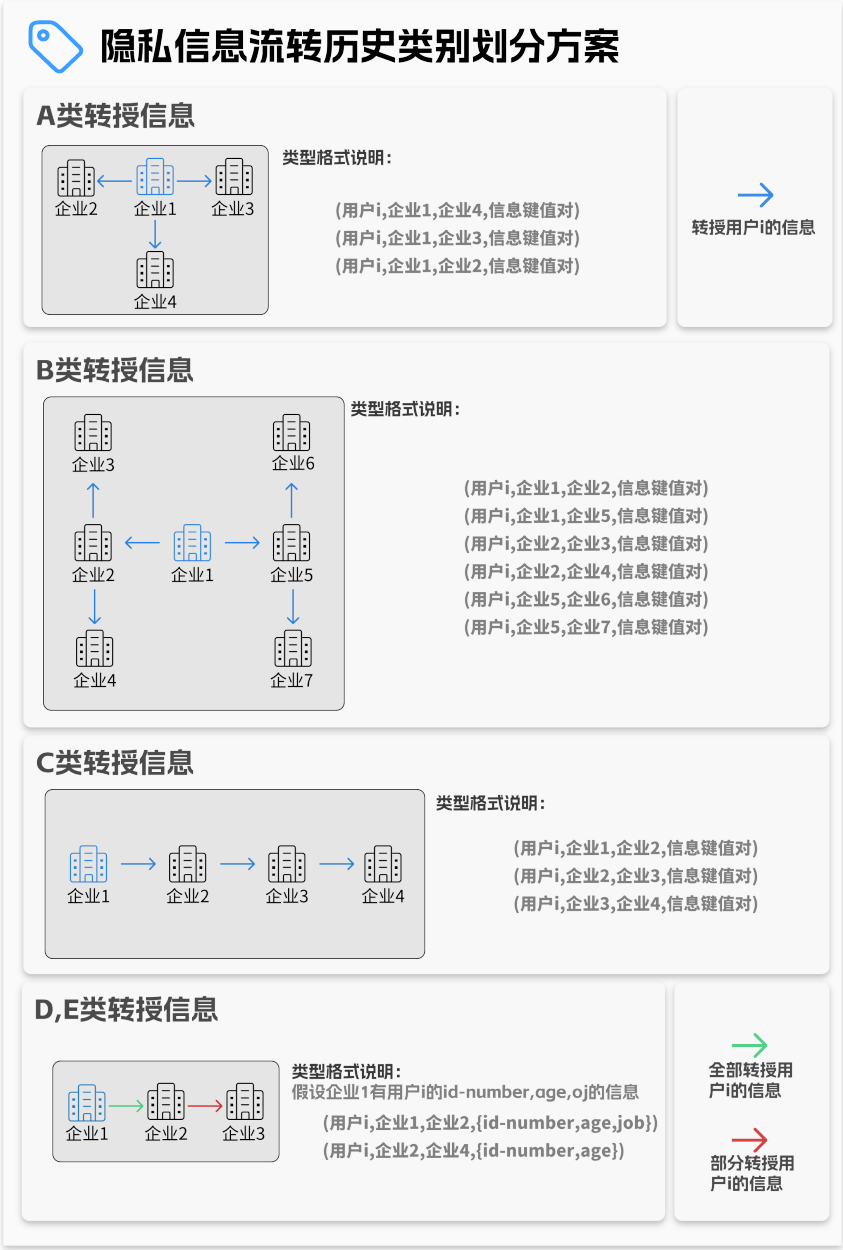


表 5.1 八种基本测试用户类

| 用户类 | 授权信息数目 | 拓扑结构  | 信息转授量 |
|-----|--------|-------|-------|
| A 类 | 低于 N   | 仅一层结构 | 完整集   |
| B 类 | 高于 N   | 树形结构  | 完整集   |
| C 类 | 低于 N   | 长链结构  | 完整集   |
| D 类 | 高于 N   | 长链结构  | 完整集   |
| E 类 | 低于 N   | 仅一层结构 | 子集    |
| F 类 | 高于 N   | 树形结构  | 子集    |
| G 类 | 低于 N   | 长链结构  | 子集    |
| H 类 | 高于 N   | 长链结构  | 子集    |

### 5.3.1.2 测试数据生成

鉴于难以从真实互联网环境中直接获取大量可用于测试的隐私信息流转情况，本数据测试方案计划通过编写 Java 脚本，基于前述用户类定义规则，生成大量的隐私信息流转用例，以供测试使用。脚本生成测试数据的流程图如下：

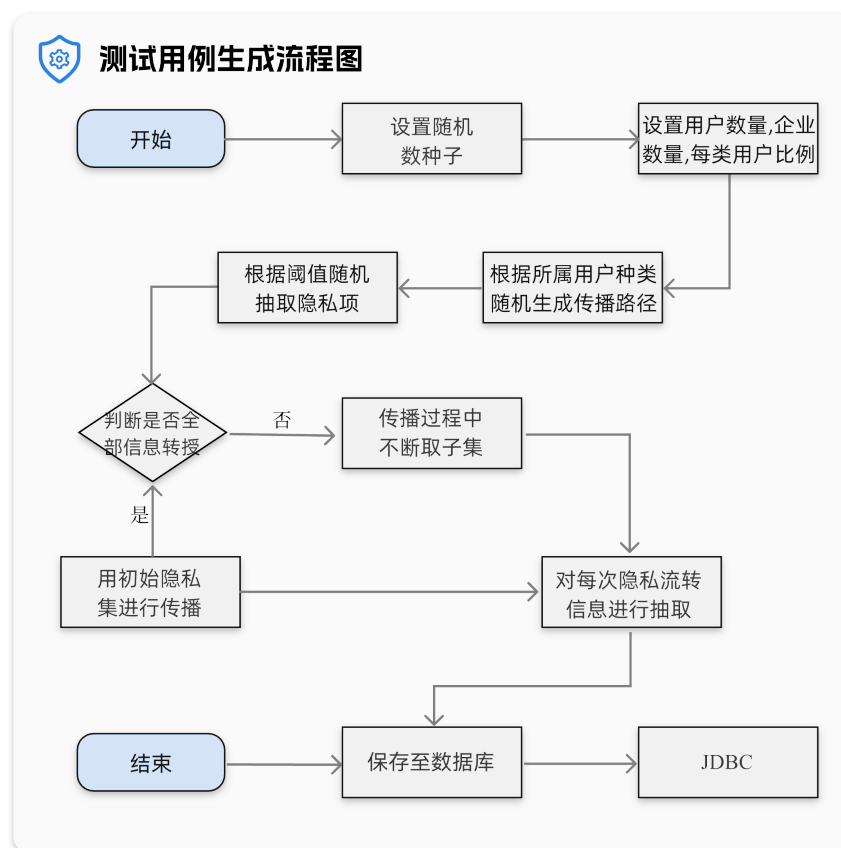


图 5.3 隐私信息流转测试用例生成流程图

为了模拟隐私信息的流转情况，测试数据生成方案将用户个人隐私分为 id number, gender, height, age, nation, phone, blood type , occupation, education, marriage, native place, current address 总计 12 类。依据设置的阈值为不同用户类随机选取隐私项，而后遵照用户类定义规则，利用随机算法生成传播路径。路径中的每个节点代表一个数据控制者，随后判断信息转授量是否为子集，随机选取子集进行转授。每项具体的隐私流转历史保存到数据库中时，应包含以下信息：用户 ID、键值对集合、上一跳控制者 ID、目标控制者 ID 以及源数据控制者 ID。

为了更加贴合真实删除场景，数据测试方案设置了 100 个数据控制者节点 (其数据控制者 ID 从 b1000 到 b1099)，同时设置了 10000 名用户 (其用户 ID 从 u1000000000 到 u1000099999)。对应于前述八种基本测试用户类，测试方案将用户均分为 8 组，每组 1250 名用户。测试生成总用例共含有 588533 项隐私转授信息，总计 320804 棵隐私信息授权树。

5.3.2 性能测试

为评估删除通知可信分发系统在实际投入使用时的性能，本次毕设设计了一系列实验。**CPU 利用率与内存占用率是反映系统运行状态最为直观的指标**，也是本次测试重点关注的两大关键指标。与此同时，系统实际为用户提供服务，从用户角度出发，还需测试系统应对**大规模删除请求时的表现（删除成功率）以及相应平均删除延时**，旨在体现系统在高并发场景下正确处理删除请求的能力。

删除通知可信分发系统的部署涉及服务器与 PC 双端，并采用了拓扑进行仿真模拟。性能测试过程中的硬件与软件配置环境如下：

表 5.2 系统测试配置表

| 类型     | 配置                                         |
|--------|--------------------------------------------|
| 机器型号   | Precision 7920 Tower                       |
| 系统版本   | Linux version 5.15.0-73-generic            |
| CPU 型号 | Intel(R) Xeon(R) Silver 4214 CPU @ 2.20GHz |
| CPU 核数 | 48 核                                       |
| 软件环境   | Python 3.7.16, Flask 2.2.3, Pypbc 0.2、     |
| 内存容量   | 128G                                       |

性能测试使用 JMeter 与 Linux 内置资源管理工具进行。JMeter 是一款基于

Java 的开源测试工具，支持协议分发所用到的 HTTP 协议，能够模拟多个用户同时对目标系统进行操作，以模拟实际的用户并发访问，从而测试系统在不同负载下的性能表现。Linux 中的进程管理 ps 命令是 Linux 内核中的常用命令，用于查看系统中运行进程的详细信息，它提供了一种查看进程状态、资源占用情况以及进程间关系的方式，进而协助用户了解系统中的进程资源消耗情况。

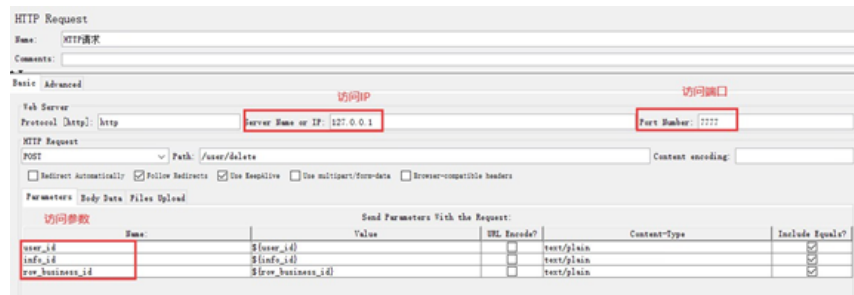


图 5.4 JMeter 生成测试脚本界面

参见图 5.4，展示了 JMeter 生成测试脚本界面。设置访问协议为 HTTP，由于测试脚本与 Web 服务器运行在同一台主机上，因此设置服务器名称 (Server Name) 为回环地址，同时将 Flask 服务器开启端口置为 7777 端口，设置请求方式为 Post 请求，需包含用户 ID、转授信息 ID、源数据控制者 ID 字段。为设置以上参数，程序将从对应的 CSV 文件中读取预先准备好的数据。在 CSV 文件中，提前预置了 800 次删除通知所需的数据。通过前述设置，使用 JMeter 来模拟并发送 800 个 POST 请求到指定的 Web 服务器，以评估系统在负载情况下的性能和可靠性。

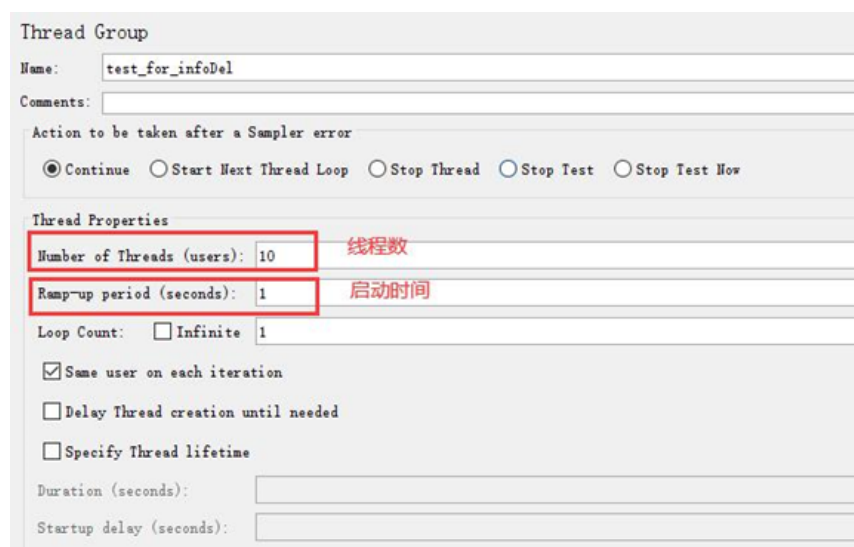


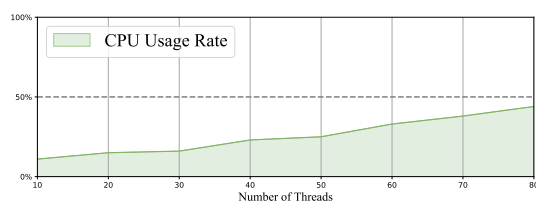
图 5.5 JMeter 线程测试配置界面

参见图 5.5，展示了 JMeter 线程测试配置界面。于 Number of Threads 属性处

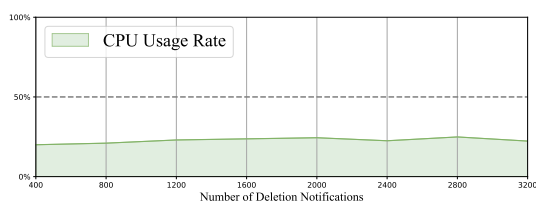
设置测试所需的线程数，并在 Ramp-up period 属性中设置线程启动时间。此处统一将时间预设为 10 秒。最后将生成的测试脚本放置于服务器上，使用 JMeter 运行，以模拟系统在并发负载下的性能。

性能测试的结果显示在图 5.6 中。在每个子图中，X 轴表示在删除通知数量固定的情况下进行删除操作所使用的线程数或固定线程数情况下接收并处理的删除通知数量。Y 轴分别表示 CPU 利用率，内存占用率，平均删除时延以及删除成功率。

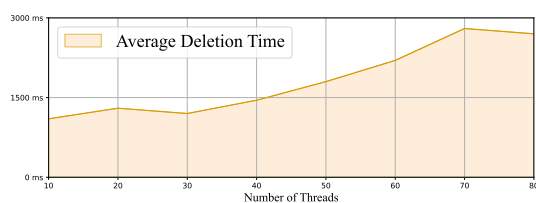
首先，参见 5.6(a)与 5.6(c)，在系统处理的删除通知数量为常量时，随着系统进行删除操作所使用线程数的增加，观察对应 CPU 利用率，内存占有率，平均删除时延以及删除成功率的变化情况。CPU 利用率，内存占用率与平均删除时间随着线程数的增加而线性缓慢增长，这主要是因为针对删除通知的操作大多为 I/O 密集型操作，需要 CPU 进行处理的时间较短。当线程数较小，并发情况不多时，系统具有较高的删除成功率，当线程数进一步增长时，系统删除成功率产生小幅波动。但总体删除通知成功率一直维持在较高水准。



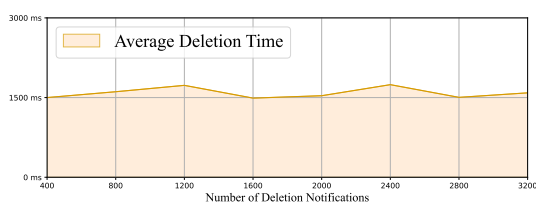
(a) 不同线程数下的 CPU 利用率与内存占用率



(b) 不同删除数量时的 CPU 利用率与内存占用率



(c) 不同线程数下的删除成功率与平均删除时延



(d) 不同删除数量时的删除成功率与平均删除时延

图 5.6 系统测评性能表现图

其次，参见 5.6(b)与 5.6(d)，在系统参与处理删除通知的线程数固定时，随着处理删除通知总量的增加，观察对应 CPU 利用率，内存占有率，平均删除时延以及删除成功率的变化情况。当需处理的删除通知总量增长时，CPU 利用率与平均删除时间无显著波动，这主要是因为处理任务多为 I/O 型任务，致使 CPU 利用率，平均删除时间与线程数呈现强相关关系。同时，删除通知数量的进一步增长对删除成功率的影响依然有限。然而，内存占用率受删除通知总量变化的影响较为明显，猜测是由于读取内存的存储数据结构的生长，致使系统需要不停读取不同的数据结构所致。

综上所述，通过对系统的性能测试结果进行分析，在用户发起删除通知请求，数据控制者接收到删除通知到最终返回用户删除凭证的整个过程中，删除成功率无显著波动，所消耗总时间的变化幅度非常小，这表明系统在真实删除情景下，面对大规模的用户删除并发请求，具有良好的处理性能与稳健性。

### 5.3.3 系统功能测试

在系统功能测试部分，为了验证隐私信息可视化处理平台的正常运行，本次毕设将使用前端界面分别对用户端，企业端与中心监管机构端的功能进行详尽测试，以确保其运行正确性。



图 5.7 首页 UI 展示图

图 5.7展示了删除通知可信分发系统的首页。用户通过浏览器开启 Web 页面后，首先进入到欢迎页面，该页面旨在帮助用户建立对系统的初步认知。使用鼠标滑轮滚动界面即可进入特色说明功能页面。

5.3.3.1 用户操作端

此系统部署于由用户，数据处理者（企业）与中心监管机构这三方所构成的信息流转生态体系下。该部分侧重于模拟用户进行隐私信息授权与删除的全过程。针对现实生活中用户的真实删除情景，本系统设置了隐私授权、隐私删除与隐私查询三大业务模块。



图 5.8 用户登录系统页面

图 5.8展示了用户登录系统页面。旨在核实用户的身份。前端会进行 Form 表单校验，输入相关信息后，点击登录按钮，将请求提交给服务器。随后便会跳转到业务选择页面，方可继续进行其他业务操作。

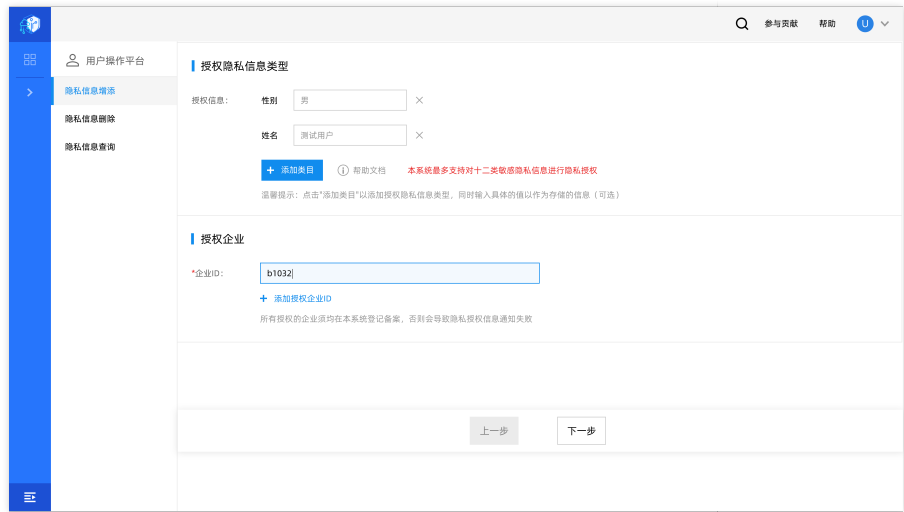


图 5.9 隐私信息授权页面

图 5.9展示了隐私信息授权页面。用户可依据自身需求，提交需要进行授权的隐私信息类型，隐私信息值（例如：身份证号：640102XXXXXXXXXXXX），同时

选定一家或多家目标授权数据控制者，输入该数据控制者对应数据控制者 ID（例如：微信-b1001）。要注意，所选择授权的数据控制者必须是已在中心监管机构处登记过后的注册数据控制者。点击下一步后，系统会通知相关数据控制者，并记录相关信息，完成隐私数据授权工作。

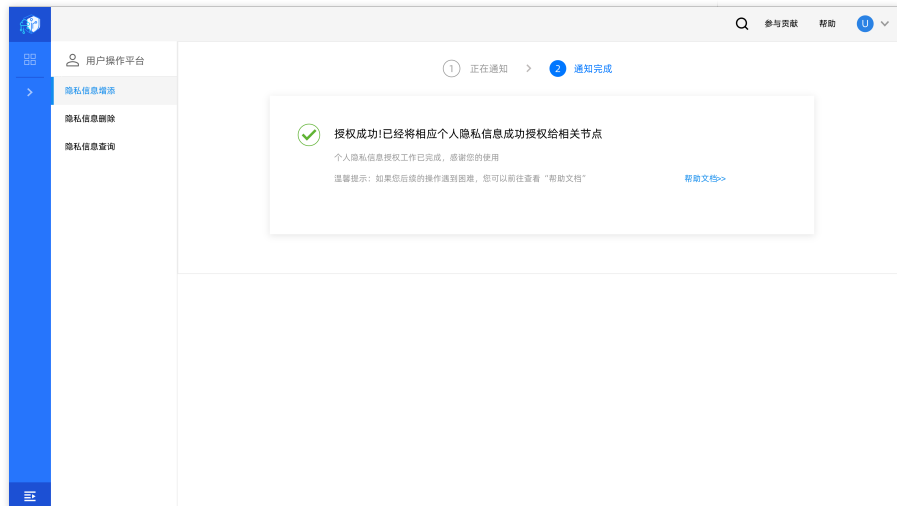


图 5.10 隐私信息授权完毕通知页面

图 ??展示了隐私信息授权完毕通知页面。这是隐私信息授权工作的最后阶段，此时会展示隐私信息授权的结果。用户可以选择点击“下载授权证书”以查看并浏览通知情况，或点击“返回业务选择界面”以选择处理其他业务。若遭遇浏览器报错等意外情况，可以选择点击“帮助文档”以获取帮助建议。

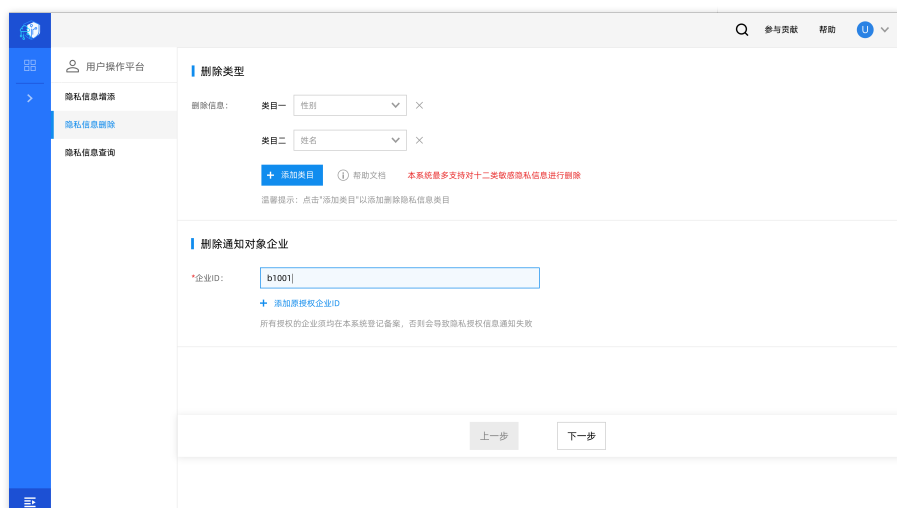


图 5.11 隐私信息删除页面

图 5.11展示了隐私信息删除页面。该部分涉及到了此系统的核心算法。点选下拉菜单来选择需要删除的隐私信息类型。同时选定接收隐私删除请求的对应数

据控制者。点击下一步按钮进行提交，此系统的删除通知分发算法会连带通知该数据控制者以及该数据控制者所连带授权的其他下属数据控制者。达成“应删尽删”、跨域删除处理的研究目的。

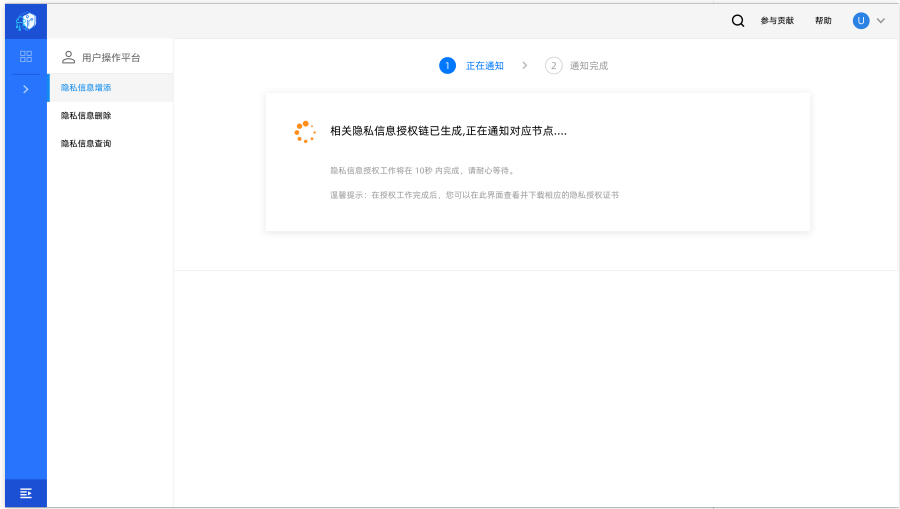


图 5.12 隐私信息删除请求正在通知页面

图 5.12展现了隐私信息删除请求正在通知页面。用户在提交了相应隐私删除请求后，便会跳转至该界面。经过 5-10 秒等待之后，系统会展示进行隐私信息删除的最终结果。

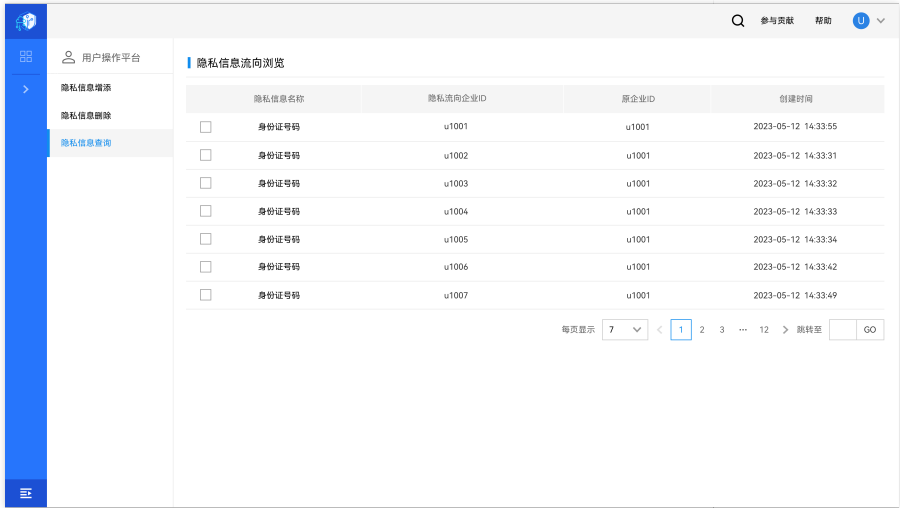


图 5.13 隐私信息查询页面

图 5.13展示了隐私信息查询页面。用户可主动进行隐私信息查询，系统会自动遍历流转历史存证子系统，筛选出相关的流转记录。用户能够查询到使用自身隐私信息的所有数据控制者，一条隐私信息流转记录包括隐私信息类型、源数据控制者 ID、此次隐私信息授权数据控制者 ID、授权创建时间等信息。

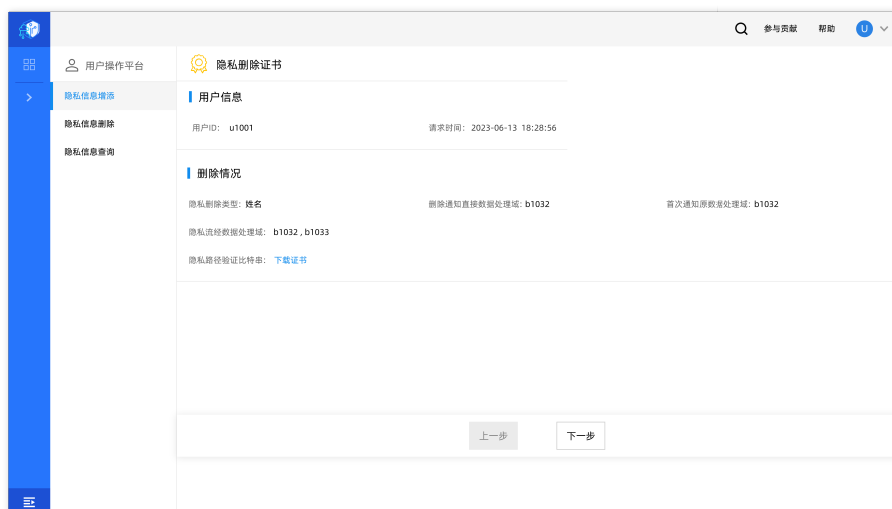


图 5.14 隐私信息删除凭证页面

图 5.14展示了**隐私信息删除凭证页面**。用户的相关隐私信息被成功删除后，便可以查看对应的隐私信息删除凭证，该凭证中包含了用户信息、删除请求时间以及凭证编号。在删除情况部分，会重点罗列出用户选择删除的隐私类型、用户选择删除的数据控制者编号以及对应信息的源数据控制者。同时罗列出隐私删除通知流转经过的对应数据控制者。最后结合删除验证部分各数据处理者的签名，生成隐私路径验证比特串，作为此次数据删除的密码学验证。

### 5.3.3.2 企业操作端

本部分侧重于模拟数据控制者使用以及转授用户个人信息。在数据控制者向中心监管机构发起转授请求后，经监管机构审核批准，便可将相关个人隐私信息转授至第三方数据处理者。在保障数据安全的前提下，进一步复用隐私数据，提高数据利用效率。



图 5.15 企业登录页面

图 5.15展示了**企业登录页面**。企业登录页面用于核实数据控制者身份，输入相关信息，之后点击登录按钮，便会跳转到业务选择页面，方可继续进行其他业务操作。

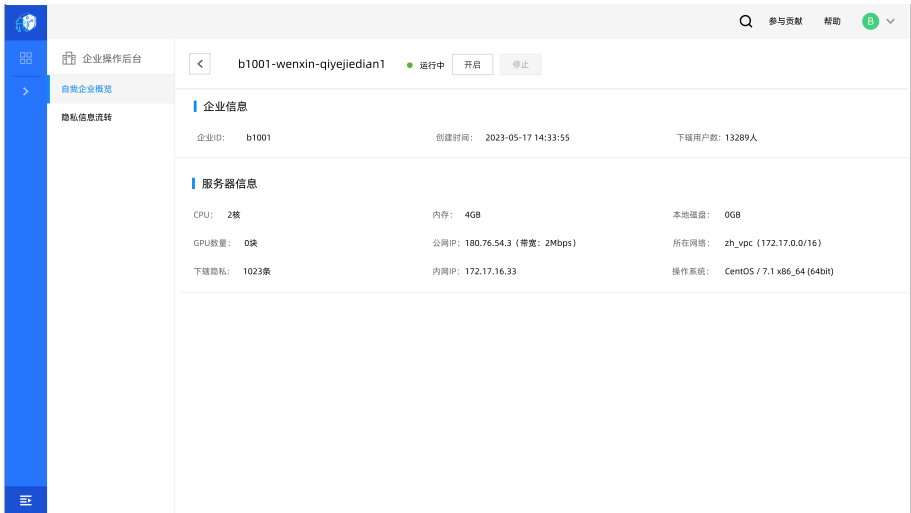


图 5.16 企业信息页面

图 5.16展示了**企业信息页面**。该部分详细罗列了数据控制者的相关信息，包括数据控制者 ID、数据控制者登记时间以及数据控制者下辖总用户数。同时服务器信息部分展示了数据控制者下辖服务器的各项参数，一个数据控制者可能会同时下辖多个服务器（此处默认一个数据控制者对应一个服务器），后续会对该部分做出进一步改进。

企业操作后台

自我企业概览

隐私信息转授

1 填写转授信息 > 2 显示转授结果

进行信息转授

\*转授企业ID: b1033

+ 添加转授企业ID

所有进行隐私信息转授的企业必须均进行过登记, 未被提前登记的企业ID无法进行信息转授 [登记入口>>](#)

\*源企业ID: b1032

\*转授用户ID: u1001

转授信息类型: 性别

若遇到问题, 点击查看 [帮助](#) >

上一步 下一步

图 5.17 企业隐私信息转授页面

图 5.17展示了**企业隐私信息转授页面**。数据控制者输入被转授的下一跳数据控制者 ID，同时输入源数据控制者 ID（隐私信息首次授权的数据控制者）。与此同时，数据控制者输入目标用户 ID 与需要转授的隐私信息类型，点击下一步，之后系统会将这些信息自动打包，提交给中心监管机构以供审核，审核通过后便会进行隐私信息转授。后续会进一步开发对应 API 接口，供自动化程序运行，以更加贴合数据控制者的实际需求。

企业操作后台

企业信息概览

隐私信息转授

1 填写转授信息 > 2 显示转授结果

转授成功! 已经将相应个人隐私信息成功转授给相关节点

个人隐私信息转授工作已完成

温馨提示: 如果您后续的操作遇到困难, 您可以前往查看“帮助文档” [帮助文档>>](#)

上一步 下一步

图 5.18 企业隐私信息转授结果页面

图 5.18展示了**企业隐私信息转授结果页面**。这是数据控制者进行隐私信息转授的最后阶段，此时会展示删除通知可信分发系统进行信息转授的最终结果。若遭遇浏览器报错等意外情况，可选择点击“帮助文档”以获取更多帮助建议。

5.3.3.3 中心监管机构端

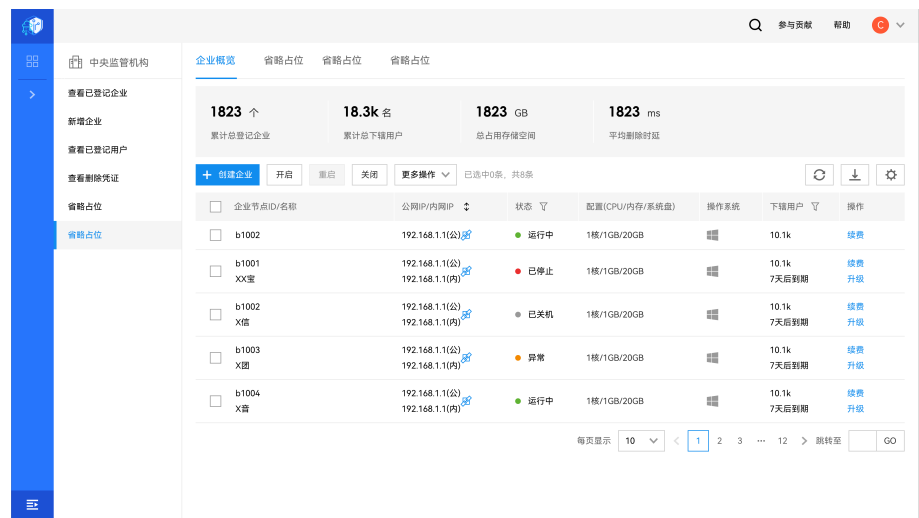


图 5.19 中心监管机构已登记企业展示页面

图 5.19展示了中心监管机构已登记企业展示页面。中心监管机构能够统计并展示所有已知数据控制者节点及其下辖服务器信息。同时可查询到系统全局下累计总登记企业数，累计总下辖用户，总占用存储空间与平均删除时延等关键信息。



图 5.20 中心监管机构新增企业页面

图 5.20展示了中心监管机构新增企业页面。中心监管机构可选择添加新企业，输入企业名称，设置好相应的密码，选择基本的服务器配置，之后便可以在中心监管机构已登记企业展示页面中查看新增企业机构及其详细信息。

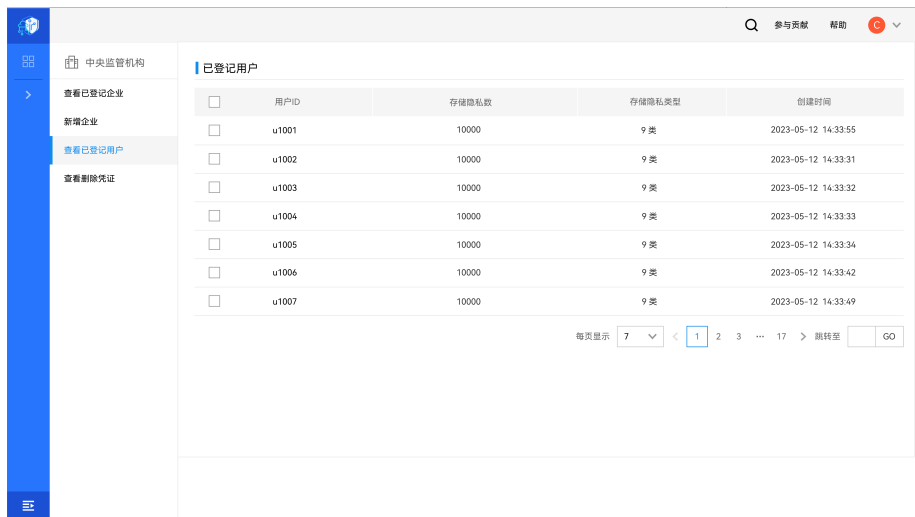


图 5.21 中心监管机构已登记用户页面

图 5.21展示了中心监管机构已登记用户页面。中心监管机构可查看所有的已注册用户，能够逐一浏览其隐私存证数目、隐私存证种类以及账户创建时间。

5.3.3.4 删除通知分发流程性测试

在本小节，以隐私删除可视化平台为例，测试删除通知可信分发系统各项基本功能的功能正确性。

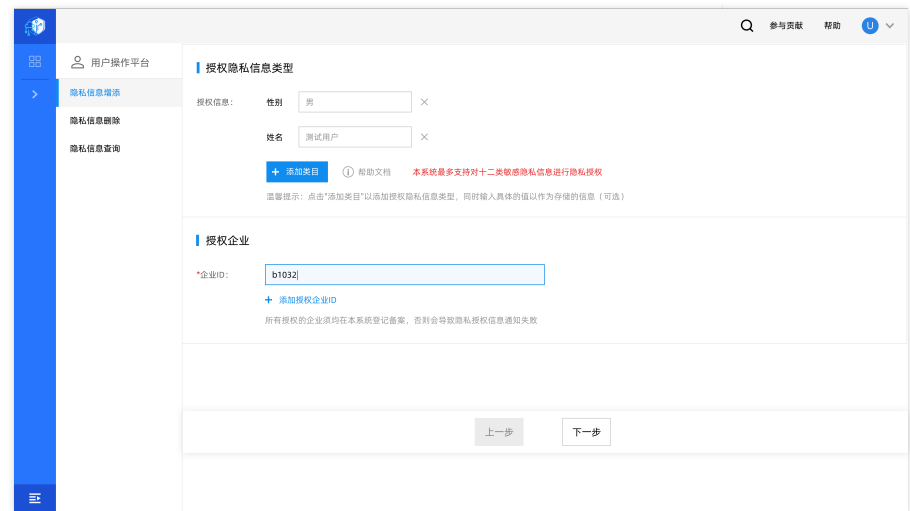


图 5.22 隐私信息转授测试

参见图 5.22，进行**隐私信息转授功能性测试**。本测试旨在测试用户端隐私信息授权的功能正确性。首先在后台注册了用户 ID 为 u1001 的账户，使用该用户账号向企业 b1032 发起隐私授权，向该企业授予用户 u1001 的相关隐私信息。

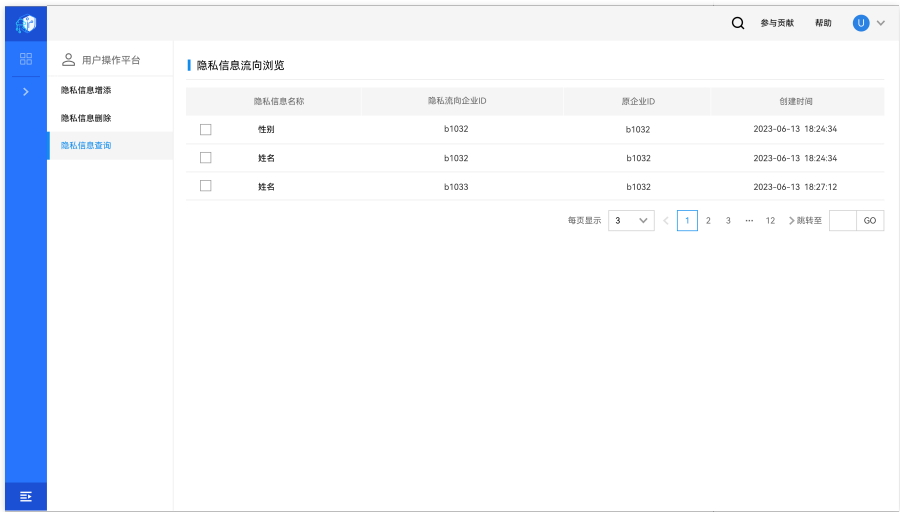


图 5.23 隐私信息浏览测试

参见图 5.23，进行**隐私信息浏览功能性测试**。该测试旨在测试隐私信息流转历史存证子系统的可靠性，同时检验前后端耦合是否良好。登录用户 ID 为 u1001 的账户，查看该账户相关隐私信息流向。



图 5.24 隐私信息删除测试

参见图 5.24，进行**隐私信息删除功能性测试**。该部分涉及了本作品的核心模块，用以验证删除通知与确认系统的功能正确性。我们登录用户 u1001 的账号，选择删除隐私信息。點選删除性别隐私信息，通知删除对象企业为 b1032。

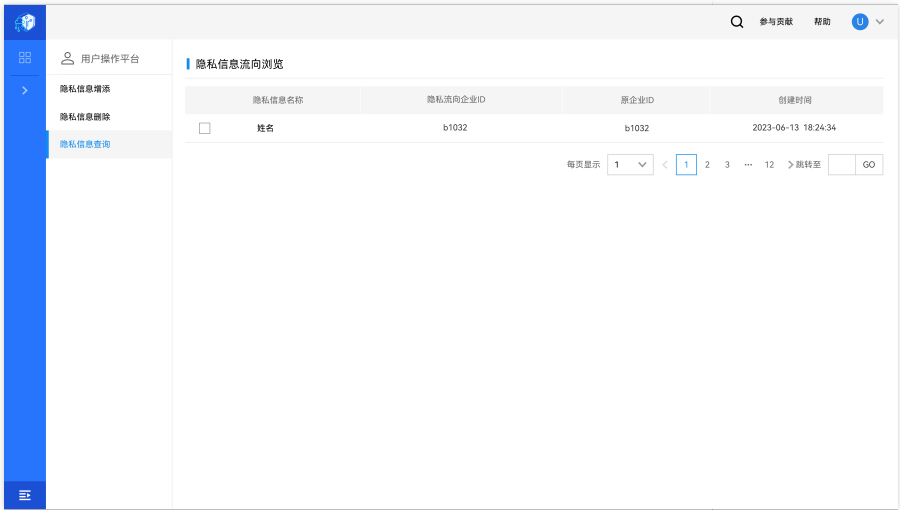


图 5.25 隐私信息状态查看检测

参见图 5.25，进行**隐私信息删除测试**。浏览隐私信息，可以发现上一步的相应删除请求已被正确执行。此时仅存有用户 b1001 向企业 b1032 授权的姓名隐私。用户 u1001 的相关性别隐私已被完全删除，企业 b1032 向企业 b1033 转授的隐私内容也被连带删除，基本实现了隐私信息跨域流转删除的研究目的。



图 5.26 删除凭证生成测试

参见图 5.26，展示了**删除凭证生成测试**。选定好测试使用的隐私信息删除流转树后，将其上传到系统中，随后运行各个功能模块，获取到运行结果后，生成对应的隐私信息删除凭证。此时用户可查看生成的隐私删除密码学凭证。其中包含了隐私删除的用户 ID，删除请求时间等重要信息。删除通知流转经过的每个数据控制者节点都会进行签名，相关签名与其他参数共同生成隐私路径验证比特串，以使用户进行下载查看。

通过浏览最终生成的删除凭证，比对用户删除凭证的各项参数，观察到凭证中记载的路径与系统运行时转发的路径一致。

## 第六章 总结与展望

本章对本次毕设所做的工作进行了总结，指出了其所做出的贡献以及局限性，同时对未来的研究工作进行了展望。

### 6.1 总结

在如今个人数据广泛跨域流通的信息时代，跨域信息共享在推动行业创新的同时，也带来了极大的隐私安全风险，隐私泄露事件屡见不鲜。对此，近年来世界各主要经济体立法机构相继颁布有关法案，旨在保障用户的隐私信息删除权。然而，产业界现有删除系统大多采用人工方式对用户删除请求进行认证与合规性审核处理。往往存在诸如删除渠道偏僻难寻，手动申请流程繁琐，**无法连带删除企业跨域授权给第三方数据控制者的相关隐私信息**，删除内容不合法律规定等问题。而在技术层面，**确保涉及的数据控制者之间完整而安全地分发删除通知是保护个人删除权利的关键**。删除权在相关理论层面的发展远远滞后于立法层面的进度，公民在实际生活中的删除权依然难以得到保障。

本毕设设计并实现了一种轻量级的删除完备通知机制——**跨域删除通知可信分发协议**，旨在保证删除通知在不可信网络中分发的完备性与可信性。该协议采用了前向遍历式验证机制，反向递归式确认机制，将删除通知可信分发问题形式化为个人信息流转树遍历验证问题，能够确保所有处理域都能收到删除请求。相关数据处理域通过协议依次按照个人信息流转树所确定的路径进行转发，并对协议中的标签进行自适应抽样验证，以减小数据处理者的开销。该协议部署于基于可信执行环境 (TEE) 的删除通知分发网络功能模块，旨在提高协议的安全性，并最大限度地减少在正常执行状态和隔离执行状态之间切换所带来的资源消耗。

在此基础上，本毕设基于该协议进一步开发了一套**删除通知可信分发系统**，旨在为产业界提供一套删除权保障技术架构。该系统由三大子系统构成：分别为隐私信息可视化平台 (前端可视化子系统)，删除指令通知与确认系统以及隐私数据流转状态管理与存证系统 (两种后端支持子系统)。隐私信息可视化处理平台为系统的前端部分，侧重于实现用户端，企业端与中心监管机构端的交互界面，旨在更好地模拟删除通知可信分发的完整业务流程。而这两种后端支持子系统旨在保障删除过程的可信性与完备性，同时为系统提供删除意图解析能力与高并发处理能

力，以应对大规模访问请求下的真实删除场景。

## 6.2 展望

尽管本次毕设在删除通知可信分发问题上取得了一定的进展，但仍存在以下几个方面不足，留待未来工作加以改进。

1. **尚未考虑监管机构单点故障问题：**上述方案以中心化监管机构作为数据的单一集中存储点，有助于建立统一的数据授权与监督秩序，还可通过集中资源与服务，实现规模效益，降低维护公民删除权的成本。然而一旦该机构受到黑客攻击、数据泄露或技术故障，将造成大规模的隐私泄露和数据风险。
2. **尚未考虑数据控制者不可信的情况：**上述方案均基于一种假设，即数据控制者均被认为是可信的，同时假设数据控制者自身倾向于维护其客户的删除权。然而，隐私数据具有一定的商业价值，不乏数据控制者铤而走险，肆意收集和共享用户个人信息。尚未探讨在数据控制者不可信情况下的应对策略。
3. **尚未形成整体性删除解决方案：**当前工作聚焦于保障删除通知在不可信数据控制者网络间分发的可信性与完备性。受限于现有知识水平，对于删除操作的可验证性与效果评估尚未进行深入研究，因此尚未形成集删除指令生成，删除通知转发与确认以及删除效果验证于一体的整体性删除解决方案。

## 致 谢

时光如梭，转眼之间，我即将结束四年的本科生涯。这段致谢不仅是对我毕业设计的总结，更是对这四年岁月的回望。站在这一重要的节点之上，回首这四年蹉跎时光，我的内心更是涌动着复杂的情绪。

首先，我谨向西安电子科技大学网络与信息安全学院的宋祁朋老师表示由衷的感谢。在这两年的时光里，十分感谢宋老师对我的悉心培养。宋老师是我在学术道路上的引路人，他引导我顺利入门并展开科研工作，始终以鼓励和支持的态度对待我，并为我提供了宝贵的机会与资源，也让我在学习过程中逐渐成长。这两年来，我做成了很多有意义的事情，也领悟了许多人生哲理。在进行毕业设计的过程中，宋老师同样耐心地为提供了指导，在成文阶段提供了许多宝贵的建议，感谢宋老师在毕业设计工作中提供的支持与帮助。

同时，感谢西安电子科技大学网络与信息安全学院的张宁老师，在本科阶段，张老师作为我们实验班的大家长，在学习生活以及升学方面为我们提供了很多宝贵的建议。感谢我们的辅导员何良老师，感谢他在本科期间对我们无微不至的照顾。感谢所有曾教导过我们的老师，他们用心传授知识，引导我们探索学术领域，培养了我们的学术素养和创新能力。正是他们的辛勤付出与教诲，让我们打下了坚实的专业基础。感谢答辩小组的李玥，李娜以及梁琳琳老师，感谢老师们在毕业设计答辩中所提出的宝贵意见，正是因为老师们的指导与建议，我才能够避免许多弯路，顺利完成了毕业设计项目与论文的工作。

我还想感谢我的学长曹静博与邓鑫，他们在我的学习与生活过程中提供了许多帮助与建议，与我分享了研究生生活中美好的点点滴滴，感谢他们所给予的对我人生规划的建议。感谢学姐王瑞芸，感谢王瑞芸学姐在毕设过程中对我的帮助与指导。感谢我的朋友董智豪，晏毅恒，陈思远，吕旋以及叶振雷，感谢实验班大家庭里其他与我相知相识的好朋友，他们都是技术能力很强的代码高手，与他们并肩拼搏，奋斗过的无数日夜，都是我大学生活里珍贵的回忆。感谢我的室友牛晨阳，张嘉伟，肖正阳，盖博以及董帧。我们一起度过了有趣而充实的四年生活，有过矛盾，也有过欢乐，这都是我青春时期最美好的回忆。

此外，我还要感谢我的父母与家人。是你们在学业和生活中给予了我无私的支持与鼓励。你们的陪伴和关心是我坚持不懈的动力，是我战胜困难的源泉。无

论是在学习上还是在生活中，你们始终与我同行，给予我无尽的爱和关怀。我将永远感激你们对我的支持和鼓励。

最后，我还想聊聊我自己。必须要承认，我的四年大学生活确实是充满了遗憾。大学生活的头两年，基本都是沉浸在恋爱，翘课和游戏当中。本来计划 Allin 作品赛，但结果并没有取得很好的成绩。回过头来冷静思考，我发现其实有很多选择和机会，但是我一直缺乏动力去追求它们，也没有下定决心去行动。也因此浪费了很多宝贵的时间与机会。

有时候，我也在思考，这结果要多好才算好啊。也许人生是旷野，而不是轨道。有的时候，我们做的事情不见得那么有意义，但重要的是自己寻找其中的意义。生活就像是一场旅程，重要的是不断地思考、探索和学习，进而不断地提升自己，让人生的旅程更加充实和有意义。

美好的大学四年即将结束，固有千般遗憾，万般不舍，我们都要怀着乐观与勇气去迈向新的起点，凡是过往，皆为序章。未来，希望我自己能够永远保持一颗谦卑、善良的心去探索未知，征服未知，不断成为更好的自己。希望所有为我提供过帮助的人都能身体健康，工作顺利。愿大家都有一个光明的未来。

## 参考文献

- [1] Alessandro, Mantelero. The EU Proposal for a General Data Protection Regulation and the roots of the ‘right to be forgotten’[J]. Computer Law & Security Report, 2013.
- [2] MYERS K, LIVELY H, ANDREWS C. New Laws Bring Much Tougher Data Protections: The California Consumer Privacy Act and the EU’s General Data Protection Regulation Grant More Rights to Consumers and Impose Obligations on Businesses[J]. Journal of Accountancy, 2019, 228.
- [3] CHEN J, SUN J. Understanding the Chinese Data Security Law[J]. International Cybersecurity Law Review, 2021, 2(2): 209-221.
- [4] 程啸. 论我国个人信息保护法中的个人信息处理规则[J]. 清华法学, 2021, 15(3): 19.
- [5] XIONG J, CHEN L, BHUIYAN M Z A, et al. A secure data deletion scheme for IoT devices through key derivation encryption and data analysis[J]. Future Generation Computer Systems, 2020, 111: 741-753.
- [6] BARTUSEK J, KHURANA D, POREMBA A. Publicly-verifiable deletion via target-collapsing functions[C]//Annual International Cryptology Conference. 2023: 99-128.
- [7] BOURTOULE L, CHANDRASEKARAN V, CHOQUETTE-CHOO C A, et al. Machine Unlearning[C]//IEEE Symposium on Security and Privacy. 2021.
- [8] THUDI A, JIA H, SHUMAILOV I, et al. On the necessity of auditable algorithmic definitions for machine unlearning[C]//31st USENIX Security Symposium (USENIX Security 22). 2022: 4007-4022.
- [9] REARDON J, BASIN D, CAPKUN S. Sok: Secure data deletion[C]//2013 IEEE symposium on security and privacy. 2013: 301-315.
- [10] LEE B, SON K, WON D, et al. Secure Data Deletion for USB Flash Memory.[J]. J. Inf. Sci. Eng., 2011, 27(3): 933-952.
- [11] MO Z, QIAO Y, CHEN S. Two-party fine-grained assured deletion of outsourced data in cloud systems[C]//2014 IEEE 34th International Conference on Distributed Computing Systems. 2014: 308-317.
- [12] XIONG J, LIU X, YAO Z, et al. A secure data self-destructing scheme in cloud computing[J]. IEEE Transactions on Cloud Computing, 2014, 2(4): 448-458.
- [13] YANG L, WEI T, ZHANG F, et al. SADUS: Secure data deletion in user space for mobile devices [J]. computers & security, 2018, 77: 612-626.
- [14] GNATYUK S, KINZERYAVYY V, SAPOZHNIK T, et al. Modern method and software tool for guaranteed data deletion in advanced big data systems[C]//Advances in Artificial Systems for

- Medicine and Education II 2. 2020: 581-590.
- [15] CHEN N, CHEN B. Duplicates also Matter! Towards Secure Deletion on Flash-based Storage Media by Removing Duplicates[C]//Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security. 2022: 54-66.
- [16] XUE L, YU Y, LI Y, et al. Efficient attribute-based encryption with attribute revocation for assured data deletion[J]. Information Sciences, 2019, 479: 640-650.
- [17] ZHENG D, XUE L, YU C, et al. Toward assured data deletion in cloud storage[J]. IEEE Network, 2020, 34(3): 101-107.
- [18] MA J, WANG M, XIONG J, et al. CP-ABE-based secure and verifiable data deletion in cloud [J]. Security and Communication Networks, 2021, 2021: 1-14.
- [19] TIAN J, WANG Z. Cloud data assured deletion scheme based on dynamic sliding window[J]. Peer-to-Peer Networking and Applications, 2022, 15(4): 1817-1833.
- [20] BAUER S, PRIYANTHA N B. Secure data deletion for Linux file systems[C]//10th USENIX Security Symposium (USENIX Security 01). 2001.
- [21] BHAT W A, QUADRI S M K. restFS: Secure data deletion using reliable & efficient stackable file system[C]//2012 IEEE 10th International Symposium on Applied Machine Intelligence and Informatics (SAMI). 2012: 457-462. DOI: 10.1109/SAMI.2012.6209010.
- [22] RUIZHONG DU X H, Pengliang SHI. Cloud data assured deletion scheme based on overwrite verification[J/OL]. Journal on Communications, 2019, 40(1), 130: 130. [https://www.infocomm-journal.com/ttxb/EN/abstract/article\\_168148.shtml](https://www.infocomm-journal.com/ttxb/EN/abstract/article_168148.shtml). DOI: 10.11959/j.issn.1000-436x.2019012.
- [23] YANG C, LIU Y, TAO X, et al. Publicly verifiable and efficient fine-grained data deletion scheme in cloud computing[J]. IEEE Access, 2020, 8: 99393-99403.
- [24] YANG C, CHEN X, XIANG Y. Blockchain-based publicly verifiable data deletion scheme for cloud storage[J]. Journal of Network and Computer Applications, 2018, 103: 185-193.
- [25] CONSTABLE S, VAN BULCK J, CHENG X, et al. {AEX-Notify}: Thwarting Precise {Single-Stepping} Attacks through Interrupt Awareness for Intel {SGX} Enclaves[C]//32nd USENIX Security Symposium (USENIX Security 23). 2023: 4051-4068.
- [26] KOGLER A, GRUSS D, SCHWARZ M. Minefield: A Software-only Protection for {SGX} Enclaves against {DVFS} Attacks[C]//31st USENIX Security Symposium (USENIX Security 22). 2022: 4147-4164.
- [27] KNIGHT S, NGUYEN H, FALKNER N, et al. The Internet Topology Zoo[J]. Selected Areas in Communications, IEEE Journal on, 2011, 29(9): 1765-1775. DOI: 10.1109/JSAC.2011.111002.